

Direct Flow Q-Learning

Shicheng Cao¹ Jingrui Jia¹ Wenyu Li¹ Feng Duan^{1,2} Tao Zhang³ Shengbo Li⁴

Abstract

Flow Matching shows great promise in offline reinforcement learning (RL), yet optimizing these iterative policies via Backpropagation Through Time (BPTT) is unstable. While prevailing paradigms circumvent this by distilling multi-step flows into single-step approximations, such methods may limit the benefits of iterative refinement. To avoid these sacrifices, we propose Direct Flow Q-Learning (DFQL), a streamlined framework that attains superior results by optimizing flow matching policies without BPTT or distillation. DFQL derives a surrogate objective that directly injects terminal Q-value gradients as a guidance term into each step velocity field, ensuring stable optimization while preserving iterative expressive capacity. Across 73 challenging tasks in OGBench and D4RL, DFQL achieves state-of-the-art results. Additionally, DFQL extends seamlessly to the offline-to-online setting, delivering substantial performance gains without further modification. Code is available at <https://github.com/2012060/DFQL>.

1. Introduction

Offline reinforcement learning (RL) aims to learn a policy from a fixed dataset of previously collected interactions, without additional environment interaction during training (Levine et al., 2020; Prudencio et al., 2024), which can reduce the cost and risk of online data collection, and provide a strong initialization for subsequent online interaction and fine-tuning (Nakamoto et al., 2023). This setting is particularly critical in domains such as autonomous driving (Guan et al., 2024) and robotic manipulation (Intelligence et al.,

¹College of Artificial Intelligence, Nankai University, Tianjin, China ²Tianjin Key Laboratory of Interventional Brain-Computer Interface and Intelligent Rehabilitation, Nankai University, Tianjin, China ³SunRisingAI Ltd., Beijing, China ⁴School of Vehicle and Mobility, Tsinghua University, Beijing, China. Correspondence to: Wenyu Li <liwenyu@nankai.edu.cn>.

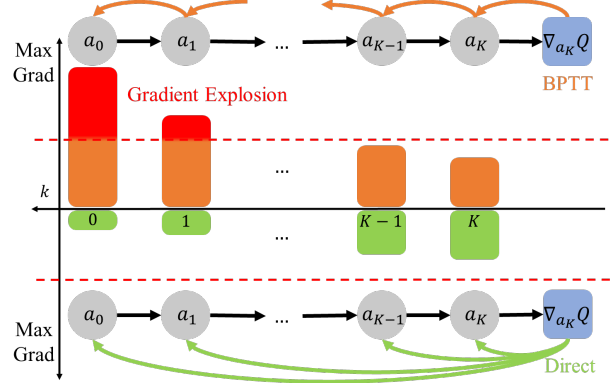


Figure 1. Flow-matching-based offline RL suffers from BPTT instability, as backpropagating terminal critic signals step-by-step through multiple discrete steps may lead to exploding gradients (orange). Hence, we propose a surrogate objective that directly injects weighted terminal Q-gradients into the velocity field at each step, bypassing this unstable recursive backpropagation path to achieve more stable training (green).

2025). Consequently, offline RL is inherently constrained by the behavior distribution reflected in the logged dataset (Koppel et al., 2024). Unlike online RL, deviations from this distribution cannot be corrected through interaction, and aggressively optimizing for a single unconstrained optimum may lead to unreliable estimates due to out-of-distribution (OOD) actions (Chen et al., 2025a). As datasets grow in scale and behavioral diversity, the induced action distributions often become highly multimodal, making the representational capacity of the policy class a critical factor for effective offline RL (Qing et al., 2024; Venkatraman et al., 2024). Motivated by this challenge, recent advances in flow matching networks, known for their strong multimodal modeling capabilities, have provided a promising foundation for expressive policy learning (Song et al., 2021; Liu et al., 2023; Lipman et al., 2023). A growing body of work has begun to integrate flow-matching-based models into offline RL, reporting consistent performance gains across diverse benchmarks (Zhang et al., 2025a; Nguyen & Yoo, 2026).

However, the integration of flow matching policies with RL objectives presents a fundamental optimization challenge. While flow matching excels at density estimation by supervising the model in the velocity (vector-field) space, RL objectives are inherently defined in the terminal action

space (Ben-Hamu et al., 2024). This discrepancy forces a difficult choice in policy extraction. On one hand, gradient-free approaches, such as value-weighted behavior cloning or critic-guided sampling (Peters & Schaal, 2007; Chen et al., 2023), avoid the complexities of differentiating through the flow steps but often rely on indirect guidance, which can limit the exploitation of the critic’s fine-grained landscape in high-dimensional spaces (Wang et al., 2023). On the other hand, seeking to optimize the policy via end-to-end differentiation of the multi-step generation process (i.e., BPTT) is unstable, often suffering from exploding gradients (Ren et al., 2025). Consequently, achieving a stable yet direct coupling between the iterative vector field and the RL objective remains an open challenge.

To circumvent the instability of BPTT, existing literature primarily follows two distinct directions. The prevailing paradigm involves policy distillation, which compresses the multi-step flow into a single-step approximation for standard RL optimization (Park et al., 2025b; Tiofack et al., 2025). Although achieving high efficiency, this simplification imposes a capacity bottleneck, as the reduction of multimodal trajectories into a simpler form constrains the iterative refinement process critical for complex task performance (Wang et al., 2026). Alternatively, another line of work seeks to stabilize BPTT directly by employing specialized architectures, such as gating mechanisms or sequential modeling, to facilitate gradient flow through the multi-step integration path (Zhang et al., 2025c). However, these structural modifications often entail implementation overhead and impose rigid architectural requirements, thereby limiting their versatility across diverse neural network designs. This raises a critical question: Can we leverage terminal Q-value signals to optimize flow matching policies while preserving their multi-step iterative structure, without imposing additional constraints on the underlying network architecture?

In this work, we exploit the structural properties of flow matching networks to propose Direct Flow Q-Learning (DFQL), a framework that injects RL objectives directly into the velocity space. Instead of backpropagating terminal Q signals through the multi-step integration path, DFQL distributes the Q gradients across all generation steps as step-wise supervision (Figure 1). We provide a theoretical analysis establishing our method as a stable approximation to BPTT. By omitting components prone to triggering gradient explosion, we effectively trade precision for enhanced numerical robustness throughout the optimization process. From another perspective, our approach can be viewed as moving inference-time guidance (Jang et al., 2025) into the training objective. This ensures the model learns a vector field that is already “value-aware” without needing extra guidance at test time. By avoiding recursive gradients, DFQL ensures stable, architecture-agnostic optimization while preserving the benefits of iterative refinement. Simi-

lar to FQL, DFQL is simple to implement within standard Actor-Critic pipelines, yet consistently outperforms baselines across 73 OGBench (Park et al., 2025a) and D4RL (Fu et al., 2020) tasks. These results bridge the gap between training stability and policy expressivity, while naturally supporting further gains via online fine-tuning.

2. Preliminaries

2.1. Offline RL

In this work, we model the environment as a discounted Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, p, r, \rho, \gamma)$, where $\mathcal{S}$ and $\mathcal{A} \subseteq \mathbb{R}^d$ denote the state and action spaces, r is the reward function, ρ is the initial-state distribution, p is the transition kernel, and $\gamma \in [0, 1)$ is the discount factor. Offline RL assumes access only to a fixed dataset $\mathcal{D} = \{(s, a, r, s')\}$ collected by an unknown behavior policy, and prohibits additional environment interaction during training. The goal is to learn a parameterized policy $\pi_\theta(a|s)$ that maximizes the expected discounted return under the MDP dynamics:

$$\mathcal{J}(\pi_\theta) = \mathbb{E}_{(s_t, a_t) \sim p^{\pi_\theta}} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (1)$$

where p^{π_θ} denotes the trajectory distribution induced by ρ , p , and π_θ . To mitigate distribution shift between π_θ and the data-collecting behavior, many offline methods optimize a regularized variant of $\mathcal{J}(\pi_\theta)$ that constrains the learned policy toward actions supported by $\mathcal{D}$. A common perspective is to view this step as constrained policy extraction from an estimated value function, which can be instantiated in several ways (Park et al., 2024):

(i) Behavior-constrained policy gradient:

$$\pi = \arg \max_{\pi} \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [\alpha Q(s, a^\pi) + \log \pi(a|s)]. \quad (2)$$

(ii) Weighted behavioral cloning:

$$\pi = \arg \max_{\pi} \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} \left[e^{\alpha(Q(s, a) - V(s))} \log \pi(a|s) \right]. \quad (3)$$

(iii) Sampling-based action selection:

$$\begin{aligned} \pi(s) &= \arg \max_{a_i \sim \pi_{\text{BC}}(\cdot|s), i \in \{1, 2, \dots, N\}} Q(s, a_i), \\ \pi_{\text{BC}} &= \arg \max_{\pi} \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [\log \pi(a|s)]. \end{aligned} \quad (4)$$

Here Q denotes the state-action value function and V denotes the state value function. In practice, they are typically estimated from the offline dataset using Monte Carlo return estimation or temporal-difference (TD) learning. The hyperparameter α and the number of candidates N control how strongly the extracted policy is constrained to remain within the dataset support.

From the objectives above, we see that in (ii) and (iii), the value functions Q and V only serve to reweight behavioral samples or select the optimal action from a set of behavior cloning (BC) produced candidates. As a result, the optimization largely preserves the underlying structure of BC (note that the BC form here is written using the standard log-likelihood for clarity; our flow matching BC objective takes a different form, which we detail in section 2.2). In contrast, (i) augments behavior cloning with a direct value maximization term $Q(s, a^\pi)$. When instantiated with multi-step flow matching generation, this term raises the BPTT stability issue. Nevertheless, motivated by the empirical evidence that behavior-constrained policy gradient style extraction is often the most effective among these paradigms (Park et al., 2025b), we aim to follow (i) while addressing its BPTT difficulty in flow matching architectures.

2.2. Flow Matching and Flow Policies in RL

Flow Matching belongs to a class of generative models that transform a simple prior distribution p_0 into a complex data distribution p_1 by constructing deterministic trajectories for samples. Rather than directly modeling the density, flow matching learns a time-dependent velocity field $v_\theta(t, x)$ that induces a flow map $\phi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$, representing the evolution of a sample from its initial state $x_0 \sim p_0$ to its state x_t at time t . The velocity field prescribes the instantaneous direction of movement for each particle via the Ordinary Differential Equation (ODE):

$$\frac{d}{dt}x_t = v_\theta(x_t, t), \quad \text{subject to } x_t = \phi_t(x_0). \quad (5)$$

By integrating these local velocity vectors from $t = 0$ to $t = 1$, the model pushes the initial distribution p_0 through a sequence of intermediate states, eventually reshaping it into the target data distribution p_1 .

Formally, let $x_0 \sim p_0$ be a sample from a simple prior (e.g., a standard Gaussian) and $x_1 \sim p_1$ be a sample from the target data distribution. We use the standard linear interpolation path commonly adopted in flow matching and rectified flow (Liu et al., 2023):

$$x_t = (1 - t)x_0 + tx_1, \quad t \in [0, 1]. \quad (6)$$

The conditional velocity that drives this motion is $\dot{x}_t = (x_1 - x_0)/(1 - 0)$. Following (Lipman et al., 2023), the standard approach to training v_θ is to minimize the flow matching objective:

$$\min_{\theta} \mathbb{E}_{\substack{x_0 \sim p_0, \\ x_1 \sim p_1, \\ t \sim \text{Unif}(0,1)}} \left[\|v_\theta(x_t, t) - (x_1 - x_0)\|_2^2 \right]. \quad (7)$$

To parameterize a generative policy $\pi_\theta(a|s)$, we extend the flow matching framework from the general coordinate space to the action space by identifying the latent particle x_t in

Equation 5 with a time-dependent action $a_t \in \mathcal{A}$. Unlike the standard formulation, the velocity field in RL must be conditioned on the state s to account for the environmental context. Consequently, we define a state-conditioned velocity field $v_\theta(s, a_t, t)$, which learns to transport a noise particle $a_0 \sim \mathcal{N}(0, I_d)$ to a terminal action a_1 at $t = 1$. Here, a_1 represents the actual action a to be executed, effectively defining the policy $\pi_\theta(a|s)$. The action generation process is governed by the following state-dependent ODE:

$$\frac{d}{dt}a_t = v_\theta(s, a_t, t). \quad (8)$$

In practice, sampling an action entails integrating the state-conditioned velocity field along the temporal path from $t = 0$ to $t = 1$. Using a K -step Euler discretization with a uniform step size $\Delta t = 1/K$, the discrete sampling trajectory is iteratively computed as:

$$a_{k+1} = a_k + \Delta t \cdot v_\theta(s, a_k, t_k), \quad (9)$$

where $t_k = k/K$ and the final action is taken as $a = a_K$. Our goal is to optimize π_θ according to the composite objective in Equation 2. In our framework, the BC component is instantiated by the flow matching objective. However, a discrepancy arises: while the BC signal provides supervision across all t , the RL objective $Q(s, a)$ is a terminal signal evaluated only at the end of the sampling trajectory. Naively differentiating this terminal value through the recursive updates in Equation 9 via BPTT requires propagating gradients through long sequences of dependencies (Park et al., 2025b; Zhang et al., 2025c). As discussed in the section 3.1, this leads to optimization instability, necessitating a more robust way to inject Q-gradients into the velocity field.

3. Direct Flow Q-Learning

To bridge the gap between velocity-space modeling and action-space RL optimization, we introduce Direct Flow Q-Learning (DFQL). Unlike existing methods that rely on distillation or complex architectural modifications, DFQL optimizes the flow matching policy by injecting value gradients directly into the velocity field. We first formalize the optimization challenges that arise when applying standard RL objectives to iterative flow matching policies.

3.1. Instability of Terminal Q-Signal BPTT in Flow Policies

By temporarily setting aside the BC constraint and focusing exclusively on the RL objective of Q-value maximization, while incorporating the iterative generation process of flow matching, we can derive the RL-specific optimization objective as follows:

$$\max_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\substack{(s,a,r,s') \sim \mathcal{D} \\ a_0 \sim \mathcal{N}(0, I_d)}} [Q(s, a_K(s, a_0; \theta))]. \quad (10)$$

To optimize this objective, the gradient $\nabla_{\theta} \mathcal{J}$ must be propagated back from the terminal action a_K through the entire sampling chain. Applying the chain rule, we obtain:

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\substack{(s,a,r,s') \sim \mathcal{D} \\ a_0 \sim \mathcal{N}(0, I_d)}} \left[\nabla_{a_K} Q(s, a_K) \cdot \frac{\partial a_K}{\partial \theta} \right] \quad (11)$$

The term $\frac{\partial a_K}{\partial \theta}$ encapsulates the sensitivity of the final action to the policy parameters, which can be expanded recursively:

$$\begin{aligned} \frac{\partial a_K}{\partial \theta} &= \sum_{k=0}^{K-1} \left(\prod_{j=k+1}^{K-1} (I + \Delta t \cdot J_j) \right) \Delta t \frac{\partial v_{\theta}(s, a_k, t_k)}{\partial \theta}, \\ J_j &\triangleq \frac{\partial v_{\theta}(s, a_j, t_j)}{\partial a_j}. \end{aligned} \quad (12)$$

This recursive formulation reveals the gradient bottleneck: the gradient at early steps is scaled by a product of Jacobians J_j . In deep flow architectures, the term $\prod (I + \Delta t \cdot J_j)$ may grow exponentially with K , leading to numerical instability.

3.2. Our Solution

The core philosophy of DFQL is to transform the terminal RL signal into a local, step-wise supervision for the velocity field. This design follows the flow-matching principle of learning local velocity fields along a probability path, while adapting it to the RL setting where policy improvement is specified by terminal value guidance rather than explicit target actions at intermediate steps. Instead of backpropagating through the entire sampling chain, we use the terminal Q-gradient to guide the velocity at each integration step toward regions of higher value.

For a given K -step Euler-discretized integration path $\{a_0, a_1, \dots, a_K\}$, let $g = \nabla_{a_K} Q(s, a_K)$ denote the terminal Q-gradient. We aim to adjust the velocity $v_{\theta}(s, a_k, t_k)$ such that it nudges the particle in a direction that maximizes the action value. We define the ‘‘value-improved’’ (RL) velocity target as:

$$\tilde{v}_k = v_{\theta}(s, a_k, t_k) + \Delta t \cdot g. \quad (13)$$

To minimize the discrepancy between the current velocity field and this improved target, we consider the following mean squared error loss at each step k :

$$\mathcal{L}_k(\theta) = \frac{1}{2} \|v_{\theta}(s, a_k, t_k) - \text{sg}[\tilde{v}_k]\|_2^2, \quad (14)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator.

Although Equation 14 is formulated as a regression task, we show that in terms of the gradient with respect to θ , it is equivalent to a linear projection of the velocity field onto the Q-gradient. Specifically:

$$\nabla_{\theta} \mathcal{L}_k(\theta) = \nabla_{\theta} (-\Delta t \cdot g^{\top} v_{\theta}(s, a_k, t_k)). \quad (15)$$

This equivalence allows us to bypass the quadratic term and directly optimize the inner product between the velocity field and the guidance signal. We provide a rigorous derivation of this gradient equivalence in Appendix A.1.

Since the terminal gradient g is evaluated at $t = 1$, its reliability in guiding the velocity field varies across the trajectory. Intuitively, velocities at later timesteps have a more direct impact on the final action a_K . We thus introduce a temporal weight $w_k = 1/(K - k)$ to reflect this increasing importance. Summing over the K -step sampling chain, the final RL objective for DFQL is:

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{\substack{(s,a,r,s') \sim \mathcal{D} \\ a_0 \sim \mathcal{N}(0, I_d)}} \left[\sum_{k=0}^{K-1} (w_k \Delta t \cdot g^{\top} v_{\theta}(s, a_k, t_k)) \right]. \quad (16)$$

By distributing the terminal reward as step-wise velocity guidance, DFQL effectively internalizes value-awareness into the flow-matching policy without the numerical instability of BPTT.

Explanation: Why This Works?

To investigate the efficacy of DFQL, we examine the gradient of the terminal value $Q(s, a_K)$ with respect to the velocity parameters θ at an intermediate sampling step $k \in \{0, \dots, K - 1\}$. For a given trajectory, applying BPTT through the discretized path yields:

$$\nabla_{\theta} Q_{k, \text{BPTT}} = \underbrace{g^{\top} \left[\prod_{j=k+1}^{K-1} (I + \Delta t \cdot J_j) \right]}_{\text{Backpropagated Signal}} \cdot \underbrace{\Delta t \frac{\partial v_{\theta}(s, a_k, t_k)}{\partial \theta}}_{\text{Local Sensitivity}}. \quad (17)$$

The term $I + \Delta t \cdot J_j$ implies that the gradient is not merely a product of deep Jacobians but travels through an explicit identity pathway at each step. In flow matching policies, the discretization step Δt acts as a natural regularizer. For a Lipschitz-continuous v_{θ} , the per-step Jacobian $I + \Delta t \cdot J_j$ remains $O(\Delta t)$ close to I . This ensures the terminal guidance g remains a dominant signal throughout the Euler-discretized integration path even without full Jacobian propagation. DFQL effectively performs state-sensitivity pruning by setting $\Delta t \cdot J_j \approx 0$ during gradient computation. This isolates the stable identity pathway and discards the interaction terms $\Delta t \cdot J_j$, which are the primary sources of exploding gradients in long-horizon offline RL. The resulting gradient contribution from step k in the DFQL objective is:

$$\nabla_{\theta} Q_{k, \text{DFQL}} = w_k \cdot g^{\top} \cdot \Delta t \frac{\partial v_{\theta}(s, a_k, t_k)}{\partial \theta}. \quad (18)$$

While this introduces a bias by omitting inter-step dependencies, we compensate for the accumulated error by using the increasing temporal weight $w_k = 1/(K - k)$, prioritizing steps closer to the terminal evaluation where the approximation is most accurate. In Appendix A.2, we demonstrate

the asymptotic consistency of this objective, proving that as $\Delta t \rightarrow 0$, the terminal-action update direction induced by DFQL RL component aligns with the terminal guidance direction, serving as a principled first-order approximation for iterative policies.

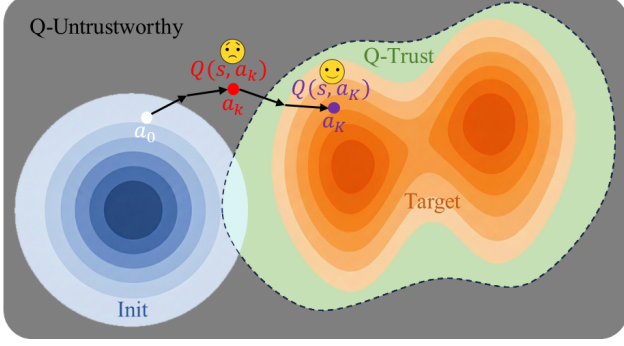


Figure 2. As intermediate actions a_k may stray far from the target distribution, the resulting $Q(s, a_k)$ becomes untrustworthy, compromising the accuracy of the value evaluation and subsequent updates.

Beyond its stability as a BPTT approximation, our objective can be viewed as moving inference-time guidance (Jang et al., 2025) into the training objective. Standard guidance techniques typically require computing $\nabla_{a_k} Q(s, a_k)$ at every sampling step to shift the trajectory, which incurs computational overhead during inference. Furthermore, in offline RL, the critic Q is trained on the data distribution of terminal actions. Evaluating its gradients on intermediate, “half-finished” particles a_t may lead to unreliable updates due to OOD states. In contrast, DFQL only utilizes the gradient evaluated at the terminal action a_K , where the critic is most accurate.

3.3. Practical Implementation and Global Objective

To instantiate the DFQL framework, we now describe the learning of the critic, the refined behavior cloning objective, and the unified optimization pipeline.

Critic learning. DFQL adopts a standard TD learning objective to train the critic. Given an offline dataset $\mathcal{D}$, we learn a parametric state-action value function $Q_\phi(s, a)$. The critic is optimized by minimizing the squared TD error:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} [(Q_\phi(s, a) - y)^2], \quad (19)$$

$$y = r + \gamma \mathbb{E}_{a' \sim \pi_\theta(\cdot|s')} [Q_{\bar{\phi}}(s', a')],$$

where $\bar{\phi}$ denotes the parameters of the target network. The expectation over next actions a' is computed by passing noise through the multi-step flow policy. Unless otherwise specified, we follow standard offline RL practices, including the use of target networks and double critics.

BC constraint. While standard flow matching uniformly samples $t \in [0, 1]$, we propose a dense BC objective to

Algorithm 1: Direct Flow Q-Learning (DFQL)

Function $\mu_\theta(s, z)$

```

 $a_0 \leftarrow z$ 
for  $k = 0, 1, \dots, K - 1$  do
     $a_{k+1} \leftarrow a_k + \frac{1}{K} v_\theta(s, a_k, t_k)$ 
return  $(a_0, a_1, \dots, a_K)$ 
    
```

while not converged do

Sample batch $(s, a, r, s') \sim \mathcal{D}; z_c, z_a \sim \mathcal{N}(0, I_a)$

$(a_0, a_1, \dots, a_K) \leftarrow \mu_\theta(s', z_c)$ $\triangleright$ Critic learning

$a' \leftarrow a_K$

Train critic Q_ϕ by minimizing Equation 19

$(a_0, a_1, \dots, a_K) \leftarrow \mu_\theta(s, z_a)$ $\triangleright$ Actor learning

$g \leftarrow \nabla_{a_K} Q_\phi(s, a_K)$

Compute BC loss $\mathcal{L}_{BC}(\theta)$ using Equation 20

Compute RL loss $\mathcal{L}_{RL}(\theta)$ using Equation 16

Compute combine loss $\mathcal{L}_\pi(\theta) = \mathcal{L}_{BC}(\theta) + \alpha \mathcal{L}_{RL}(\theta)$

Train actor π_θ by minimizing Equation 21

return policy π_θ

ensure structural symmetry with our RL objective. Since $\mathcal{L}_{RL}$ provides supervision at each discrete time step k , a sparse BC signal may lead to temporal imbalance in the supervision provided to the velocity field. We thus train v_θ on each discrete point along the straight-line bridge:

$$\mathcal{L}_{BC}(\theta) = \mathbb{E}_{\substack{(s,a,r,s') \sim \mathcal{D} \\ a_0^{\text{BC}} \sim \mathcal{N}(0, I_a)}} \left[\sum_{k=0}^{K-1} \|v_\theta(s, a_k^{\text{BC}}, t_k) - (a - a_0^{\text{BC}})\|_2^2 \right], \quad (20)$$

where $a_k^{\text{BC}} = (1 - t_k)a_0^{\text{BC}} + t_k a$ represents the interpolated particle at step k . This dense supervision ensures that the imitation and reinforcement signals operate at the same “density” in velocity space, enhancing training stability when the two objectives are combined.

Unified policy optimization. The final policy objective balances the refined BC loss with the proposed DFQL RL loss. By combining Equation 16 and Equation 20, the global optimization problem is formulated as:

$$\mathcal{L}_\pi(\theta) = \mathcal{L}_{BC}(\theta) + \alpha \mathcal{L}_{RL}(\theta), \quad (21)$$

where α is a hyperparameter that controls the trade-off between maximizing the Q-value and adhering to the data distribution. The complete training procedure is detailed in Algorithm 1, and more detailed implementation specifics are provided in Appendix C.

4. Related Works

4.1. Offline RL and Policy Extraction

Offline RL aims to derive effective policies from static datasets while mitigating performance collapse caused by OOD actions (Levine et al., 2020). This is typically achieved through conservative value learning (Kumar et al., 2020; Yu et al., 2021; An et al., 2021) or constrained policy extraction (Wang et al., 2025). As formalized in Section 2, existing policy extraction paradigms can be broadly categorized into three lines. **Behavior-constrained policy gradient** methods directly maximize the Q-function alongside a BC regularizer (Fujimoto & Gu, 2021; Tarasov et al., 2023a; Liu et al., 2024); while these methods (e.g., TD3+BC) are highly effective, they encounter significant optimization hurdles when applied to multi-step generative policies due to the need for differentiation through the generated trajectory (Fang et al., 2025). Alternatively, **weighted behavioral cloning** approaches, such as AWR or IQL, reweight dataset actions based on value or advantage estimates (Nair et al., 2020; Qing et al., 2024; Peters & Schaal, 2007). This paradigm is naturally stable and compatible with generative models but is often overly conservative as policy improvement is strictly bounded by the dataset support (Kostrikov et al., 2022). Lastly, **sampling-based selection** methods utilize the critic to rank and select candidates from multiple action proposals (Kang et al., 2023; Mao et al., 2024; Chen et al., 2023). While these are compatible with expressive policy parameterizations, they incur significant computational costs during training and inference due to the extensive sampling required (Fujimoto et al., 2019; Chen et al., 2024b). Our framework, DFQL, builds on the behavior-constrained paradigm but circumvents its optimization instability by directly injecting value gradients into the velocity field.

4.2. Flow-Matching and other Generative Policies

Generative policies are typically more expressive than simple unimodal parameterizations, enabling the modeling of multimodal action distributions. In offline settings, this can facilitate better alignment with the dataset support. Prior work has explored generative policies along multiple technical directions, including parameterizations based on latent variables (Zhou et al., 2020), autoregressive distributions (Korenkevych et al., 2019), mixture models (Zhao et al., 2022), and trajectory modeling (Chen et al., 2021; Zhao et al., 2026). Recently, iterative generative policies have attracted increasing attention, with diffusion models (Wang et al., 2023; Chen et al., 2024b; Fang et al., 2025; Mao et al., 2024) and flow matching emerging as prominent routes for bringing highly expressive generative parameterizations to offline RL.

Motivated by the simplicity of flow matching architectures, their often higher inference efficiency relative to diffusion-

based policies (Park et al., 2025b; Kang et al., 2023), and the fact that this direction remains comparatively under-explored, offline RL with flow matching policy networks has received increasing attention. Our work also falls into this category. Among existing work, a representative line, exemplified by FQL, distills a multi-step generative policy into a one-step policy, providing a stable interface for value-based policy improvement (Park et al., 2025b; Tiofack et al., 2025). Another line incorporates critic preferences as energy weighting or guidance, injecting them into the flow matching objective or the sampling procedure to drive policy improvement (Feng et al., 2025; Zhang et al., 2025a; Alles et al., 2025). Beyond these, recent studies explore path-level regularization to improve the stability of learning the vector field (Chen et al., 2025b), employ specialized network architectures such as gating mechanisms to stabilize the BPTT process (Zhang et al., 2025c), or explicitly formulate the generation process as a sequential decision problem (Zhang et al., 2025b). DFQL is most closely related to FQL, but does not rely on policy distillation. Instead, it stabilizes training via a surrogate objective and applies RL-based improvement directly to the flow matching network. Beyond purely offline RL, many studies also investigate flow matching policies in the offline-to-online setting (Shin et al., 2026; Li et al., 2025). Our experiments show that DFQL can be applied to offline-to-online RL without any modification and still meets the requirements of this setting.

5. Experiments

In this section, we evaluate our method and compare it against prior offline RL and offline-to-online RL approaches across challenging tasks. We also run ablations to examine key design choices and strategy components.

5.1. Experimental Setup

Benchmarks. We evaluate our method on two standard offline RL benchmarks, OGBench (Park et al., 2025a) and D4RL (Fu et al., 2020). Our main evaluation is on OGBench, which offers a broad collection of locomotion and manipulation tasks and supports state-based settings under a unified interface. Following the established protocol and keeping the setup consistent with FQL (Park et al., 2025b), we use the reward-based single-task variants of OGBench and report results on 50 state-based tasks. We allocate a larger portion of experiments to OGBench to assess performance across a wider range of task conditions and behavioral coverage, enabling more informative comparisons under demanding settings. In parallel, we retain evaluation on D4RL to improve comparability with prior work, focusing on the commonly reported AntMaze navigation suite and the Adroit dexterous manipulation suite.

Methods compared. In the offline RL experiments, we

Table 1. Offline RL results. Across the vast majority of environments, DFQL achieves optimal or near-optimal performance, demonstrating strong and consistent results. All numbers are reported as means over 8 seeds (4 seeds for pixel-based tasks), and the complete experimental results are provided in Table 5.

Task category	Gaussian policies			Diffusion policies			Flow policies				
	BC	IQL	ReBRAC	IDQL	SRPO	CAC	FAWAC	FBRAC	IFQL	FQL	DFQL
OGbench antmaze-large-singletask (5 tasks)	11 ± 1	53 ± 3	81 ± 5	21 ± 5	11 ± 4	33 ± 4	6 ± 1	60 ± 6	28 ± 5	79 ± 3	88 ± 2
OGbench antmaze-giant-singletask (5 tasks)	0 ± 0	4 ± 1	26 ± 8	0 ± 0	0 ± 0	0 ± 0	0 ± 0	4 ± 4	3 ± 2	9 ± 6	39 ± 4
OGbench humanoidmaze-medium-singletask (5 tasks)	2 ± 1	33 ± 2	22 ± 8	1 ± 0	1 ± 1	53 ± 8	19 ± 1	38 ± 5	60 ± 14	58 ± 5	66 ± 9
OGbench humanoidmaze-large-singletask (5 tasks)	1 ± 0	2 ± 1	2 ± 1	1 ± 0	0 ± 0	0 ± 0	0 ± 0	2 ± 0	11 ± 2	4 ± 2	8 ± 3
OGbench antsoccer-arena-singletask (5 tasks)	1 ± 0	8 ± 2	0 ± 0	12 ± 4	1 ± 0	2 ± 4	12 ± 0	16 ± 1	33 ± 6	60 ± 2	59 ± 4
OGbench cube-single-singletask (5 tasks)	5 ± 1	83 ± 3	91 ± 2	95 ± 2	80 ± 5	85 ± 9	81 ± 4	79 ± 7	79 ± 2	96 ± 1	96 ± 1
OGbench cube-double-singletask (5 tasks)	2 ± 1	7 ± 1	12 ± 1	15 ± 6	2 ± 1	6 ± 2	5 ± 2	15 ± 3	14 ± 3	29 ± 2	38 ± 7
OGbench scene-singletask (5 tasks)	5 ± 1	28 ± 1	41 ± 3	46 ± 3	20 ± 1	40 ± 7	30 ± 3	45 ± 5	30 ± 3	56 ± 2	61 ± 3
OGbench puzzle-3x3-singletask (5 tasks)	2 ± 0	9 ± 1	21 ± 1	10 ± 2	18 ± 1	19 ± 0	6 ± 2	14 ± 4	19 ± 1	30 ± 1	65 ± 7
OGbench puzzle-4x4-singletask (5 tasks)	0 ± 0	7 ± 1	14 ± 1	29 ± 3	10 ± 3	15 ± 3	1 ± 0	13 ± 1	25 ± 5	17 ± 2	25 ± 3
D4RL antmaze (6 tasks)	17	57	78	79	74	30	44 ± 3	64 ± 7	65 ± 7	84 ± 3	82 ± 3
D4RL adroit (12 tasks)	48	53	59	52 ± 1	51 ± 1	43 ± 2	48 ± 1	50 ± 2	52 ± 1	52 ± 1	52 ± 1
Visual manipulation (5 tasks)	-	42 ± 4	60 ± 2	-	-	-	-	22 ± 2	50 ± 5	65 ± 2	72 ± 5

Table 2. Offline-to-online RL results. Consistent with established benchmarks for offline-to-online RL (Prudencio et al., 2024), we employ 1M steps for both the initial offline phase and the subsequent online fine-tuning stage. Full results are presented in Table 4.

BC	IQL	ReBRAC	Cal-QL	RLPD	IFQL	FQL	DFQL
OGbench (5 tasks) ¹	8 ± 3 → 5 ± 2	17 ± 5 → 29 ± 9	0 ± 0 → 10 ± 11	0 ± 0 → 42 ± 2	23 ± 8 → 48 ± 11	40 ± 4 → 68 ± 11	48 ± 7 → 89 ± 9
D4RL antmaze (6 tasks)	56 → 78	80 → 79	53 → 97	0 ± 0 → 96 ± 1	65 ± 6 → 90 ± 2	75 ± 8 → 95 ± 5	83 ± 2 → 96 ± 1
D4RL adroit (12 tasks)	22 → 45	21 → 93	-1 → -1	1 ± 1 → 88 ± 8	21 ± 2 → 56 ± 6	13 ± 4 → 110 ± 5	17 ± 2 → 106 ± 4

¹As offline-to-online RL is not the primary focus of our study, we evaluate only five representative environments from the OGBench suite to ensure a consistent experimental setup with FQL (Park et al., 2025b).

compare against ten methods spanning three categories: Gaussian, diffusion, and flow-based approaches. The Gaussian category includes the basic BC baseline, the classic IQL (Kostrikov et al., 2022), and the strong performer ReBRAC (Tarasov et al., 2023a). The diffusion category covers IDQL (Hansen-Estruch et al., 2023) based on rejection sampling, as well as SRPO (Chen et al., 2024a) and CAC (Ding & Jin, 2024) based on policy distillation. The flow category includes representative flow-policy methods that instantiate different policy extraction strategies on top of a flow matching policy, namely FAWAC as a flow variant of AWAC (Nair et al., 2020), IFQL as a flow variant of rejection-sampling methods, and FBRAC as a flow variant of BRAC (Wu et al., 2019), together with the high-performing policy-distilled flow matching method FQL. These flow-based comparisons are designed to highlight how different policy extraction strategies behave when combined with flow matching in a straightforward manner. In particular, FBRAC is the closest baseline to our DFQL, but it relies on the classical gradient backpropagation mechanism to propagate terminal Q information through all discrete generation steps. Contrasting it with our direct guidance injection provides an ablation-style evaluation of the proposed information injection design.

Beyond comparisons with purely offline methods, we also evaluate our approach in the offline-to-online RL setting. In this study, we include three strong baselines that support

offline-to-online fine-tuning (IQL, ReBRAC, and FQL) and additionally compare against two methods specifically designed for offline-to-online RL, Cal-QL (Nakamoto et al., 2023) and RLPD (Ball et al., 2023).

Evaluation. We evaluate all methods after a fixed number of gradient update steps. To mitigate potential reporting bias, we report the final performance rather than the best score observed during training (Tarasov et al., 2023b). Results are presented as the mean and variability across multiple random seeds, values following “±” in tables denote standard deviation, while shaded areas in plots represent the 95% bootstrap confidence interval. To ensure a fair comparison, we adopt the baseline results directly from FQL (Park et al., 2025b), where hyperparameters were extensively optimized. For DFQL, we follow the same MLP backbone configuration and discount factor as in that study to ensure comparability, while any minor implementation deviations are described in Appendix C. Regarding evaluation metrics, OGBench tasks are measured by the episode success rate, while on D4RL we follow the standard protocols, using binary success rates for AntMaze and normalized returns for Adroit. Full training and evaluation details are provided in Appendix C and F.

5.2. Main Results and Analysis

Performance comparison. According to the comparative results in Table 1, DFQL achieves superior performance over both existing strong Gaussian-policy methods and diffusion-policy methods, demonstrating the effectiveness of the flow matching policy itself. Moreover, DFQL’s advantage over FBRAC indicates that the approximations adopted in our approach effectively stabilize the training process. Its improvement over FQL further suggests that retaining the multi-step generation procedure indeed leads to performance gains. Notably, the substantial improvements of DFQL on challenging tasks (e.g., “antmaze-giant-singletask” and “puzzle-3x3-singletask-task”) reflect enhanced representational capacity for complex tasks. In summary, the experimental results demonstrate that DFQL is a viable and competitive method for offline RL.

The results for the offline-to-online transition are reported in Table 2. Notably, our method achieves optimal or near-optimal performance in this phase even without incorporating specialized offline-to-online mechanisms.

Effectiveness of DFQL in mitigating gradient explosion.

To isolate DFQL’s effect on gradient explosion, we replace the terminal-gradient direct guidance with a BPTT-style step-by-step backpropagation baseline while keeping all other settings unchanged. We then evaluate training stability by tracking the maximum gradient magnitude on three environments. As shown in Figure 3, DFQL substantially mitigates the gradient explosion under BPTT, leading to a more stable optimization process. Additional ablations on other key components are provided in Appendix C.

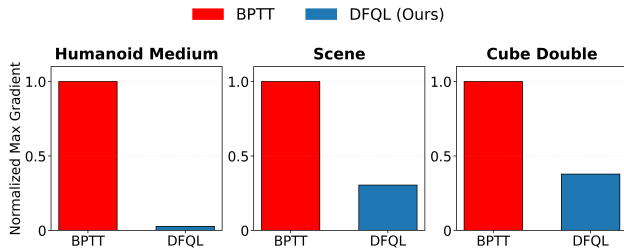


Figure 3. **Comparison of maximum gradient magnitudes.** To represent the worst-case scenario, we collect the maximum absolute gradients across 8 seeds and normalize the values within each task by scaling the largest magnitude to 1.0.

DFQL as a modular algorithmic component. Beyond its standalone performance, we investigate whether the gradient stability provided by DFQL can be combined with other orthogonal RL enhancements. To evaluate this, we integrate our method with Q-chunking (QC), a technique specifically tailored for long-horizon and sparse-reward tasks by operating within a temporally extended action space (Li et al., 2025). Specifically, QC trains the policy to predict a sequence of future actions while the critic estimates the

value of the entire sequence, promoting temporally coherent exploration and enabling unbiased n -step value backups. Crucially, DFQL and QC target different stages of the RL pipeline. Our method stabilizes the policy extraction process from flow-matching-based models, whereas QC focuses on optimizing action representation and credit assignment. This structural difference allows for a seamless integration into the QC-DFQL variant. As illustrated in Figure 4, QC-DFQL achieves a consistent performance boost over the high-performing QC-FQL (a variant combining QC with FQL) baseline across representative sparse-reward manipulation tasks. These results suggest that DFQL is a lightweight modification that can synergize with other RL architectures to further enhance performance in complex environments. We provide a further introduction of the QC-DFQL implementation in Appendix D.

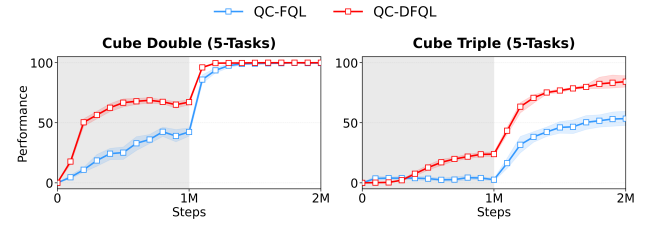


Figure 4. **Synergy between DFQL and QC.** To evaluate DFQL’s modularity, we integrate it with QC across “cube-double/triple” tasks. Results show a two-stage training trajectory: offline RL (0-1M steps) and online fine-tuning (1-2M steps), with the transition marked by the shaded boundary. QC-DFQL outperforms the QC-FQL baseline, highlighting DFQL as a compatible component for flow-matching-based RL architectures. Full results in Fig. 7.

6. Conclusion

In this paper, we presented Direct Flow Q-Learning, an approach to integrating flow matching with offline RL. By addressing the fundamental challenges of BPTT-based optimization, we provide a compelling alternative to the common distillation-centric framework. Our key contribution is the development of a surrogate objective that enables the direct propagation of RL signals into the flow matching vector field. This mechanism facilitates stable and efficient policy learning while maintaining the multi-step sampling structure, thereby avoiding the potential trade-offs associated with one-step approximations. Extensive empirical evaluations across 73 tasks confirm that DFQL not only achieves superior performance in static offline datasets but also exhibits robust adaptability during offline-to-online transition. While our work demonstrates significant advancements, a discussion regarding its current limitations is provided in Appendix B. We believe that the paradigm of direct gradient injection offered by DFQL provides a scalable and effective blueprint for future research in flow-matching-based RL.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant No. 62303244, and the Open Fund Project of the State Key Laboratory of Intelligent Green Vehicle and Mobility under Grant No. KFY260406.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Alles, M., Chen, N., van der Smagt, P., and Cseke, B. Flowq: Energy-guided flow policies for offline reinforcement learning. *arXiv preprint arXiv:2505.14139*, 2025.
- An, G., Moon, S., Kim, J.-H., and Song, H. O. Uncertainty-based offline reinforcement learning with diversified q-ensemble. In *Neural Information Processing Systems (NeurIPS)*, 2021.
- Ball, P. J., Smith, L., Kostrikov, I., and Levine, S. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning (ICML)*, 2023.
- Ben-Hamu, H., Puny, O., Gat, I., Karrer, B., Singer, U., and Lipman, Y. D-flow: Differentiating through flows for controlled generation. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *International Conference on Machine Learning (ICML)*, 2024.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. Jax: composable transformations of Python+NumPy programs. <https://github.com/google/jax>, 2018.
- Chen, H., Lu, C., Ying, C., Su, H., and Zhu, J. Offline reinforcement learning via high-fidelity generative behavior modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Chen, H., Lu, C., Wang, Z., Su, H., and Zhu, J. Score regularized policy optimization through diffusion behavior. In *International Conference on Learning Representations (ICLR)*, 2024a.
- Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. Decision transformer: Reinforcement learning via sequence modeling. In *Neural Information Processing Systems (NeurIPS)*, 2021.
- Chen, T., Wang, Z., and Zhou, M. Diffusion policies creating a trust region for offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2024b.
- Chen, T., Cai, R., Wu, F., and Zhang, X. Active: Offline reinforcement learning via adaptive imitation and in-sample V -ensemble. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Chen, X., Wang, S., and Yao, L. Continuity-regularized flow matching for offline reinforcement learning. OpenReview, 2025b. URL <https://openreview.net/forum?id=diVihFiZSc>. Submitted to ICLR 2026.
- Ding, Z. and Jin, C. Consistency models as a rich and efficient policy class for reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Fang, L., Liu, R., Zhang, J., Wang, W., and Jing, B. Diffusion actor-critic: Formulating constrained policy iteration as diffusion noise regression for offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.
- Feng, R., Wu, T., Yu, C., Deng, W., and Hu, P. On the guidance of flow matching. In *International Conference on Machine Learning (ICML)*, 2025.
- Fu, J., Kumar, A., Nachum, O., Tucker, G., and Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.
- Fujimoto, S. and Gu, S. S. A minimalist approach to offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2021.
- Fujimoto, S., Meger, D., and Precup, D. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning (ICML)*, 2019.
- Geng, Z., Deng, M., Bai, X., Kolter, J. Z., and He, K. Mean flows for one-step generative modeling. In *Neural Information Processing Systems (NeurIPS)*, 2025.
- Guan, J., Shen, L., Zhou, A., Li, L., Hu, H., He, X., Chen, G., and Jiang, C. Poce: Primal policy optimization with conservative estimation for multi-constraint offline reinforcement learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Hansen-Estruch, P., Kostrikov, I., Janner, M., Grudzien, K., and Levine, S. Idql: Implicit q-learning as an actor-critic method with diffusion policies. Technical Report UCB/EECS-2023-62, EECS Department, University of California, Berkeley, May 2023. URL <http://www2.eecs.berkeley.edu/Pubs/TechRpts/2023/EECS-2023-62.html>.

- Intelligence, P., Amin, A., Aniceto, R., Balakrishna, A., Black, K., Conley, K., Connors, G., Darpinian, J., Dhabalia, K., DiCarlo, J., Driess, D., Equi, M., Esmail, A., Fang, Y., Finn, C., Glossop, C., Godden, T., Goryachev, I., Groom, L., Hancock, H., Hausman, K., Hussein, G., Ichter, B., Jakubczak, S., Jen, R., Jones, T., Katz, B., Ke, L., Kuchi, C., Lamb, M., LeBlanc, D., Levine, S., Li-Bell, A., Lu, Y., Mano, V., Mothukuri, M., Nair, S., Pertsch, K., Ren, A. Z., Sharma, C., Shi, L. X., Smith, L., Springenberg, J. T., Stachowicz, K., Stoeckle, W., Swerdlow, A., Tanner, J., Torne, M., Vuong, Q., Walling, A., Wang, H., Williams, B., Yoo, S., Yu, L., Zhilinsky, U., and Zhou, Z. $\pi_{0,6}^*$: A vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.
- Jang, Y., Nam, H. C., Park, J. M., Bae, G., and Kwon, H. Q-guided flow q-learning. In *Conference on Robot Learning (CoRL)*, Seoul, Korea, 2025.
- Kang, B., Ma, X., Du, C., Pang, T., and Yan, S. Efficient diffusion policies for offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2023.
- Koppel, A., Bhatt, S., Guo, J., Eappen, J., Wang, M., and Ganesh, S. Information-directed pessimism for offline reinforcement learning. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *International Conference on Machine Learning (ICML)*, 2024.
- Korenkevych, D., Mahmood, A. R., Vasan, G., and Bergstra, J. Autoregressive policies for continuous control deep reinforcement learning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.
- Kostrikov, I., Nair, A., and Levine, S. Offline reinforcement learning with implicit q-learning. In *International Conference on Learning Representations (ICLR)*, 2022.
- Kumar, A., Zhou, A., Tucker, G., and Levine, S. Conservative q-learning for offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2020.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020.
- Li, Q., Zhou, Z., and Levine, S. Reinforcement learning with action chunking. In *Neural Information Processing Systems (NeurIPS)*, 2025.
- Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.
- Liu, T., Li, Y., Lan, Y., Gao, H., Pan, W., and Xu, X. Adaptive advantage-guided policy regularization for offline reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.
- Liu, X., Gong, C., and Liu, Q. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.
- Mao, L., Xu, H., Zhan, X., Zhang, W., and Zhang, A. Diffusion-dice: In-sample diffusion guidance for offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2024.
- Nair, A., Gupta, A., Dalal, M., and Levine, S. Awac: Accelerating online reinforcement learning with offline datasets. *arXiv preprint arXiv:2006.09359*, 2020.
- Nakamoto, M., Zhai, S., Singh, A., Mark, M. S., Ma, Y., Finn, C., Kumar, A., and Levine, S. Cal-ql: Calibrated offline rl pre-training for efficient online fine-tuning. In *Neural Information Processing Systems (NeurIPS)*, 2023.
- Nguyen, T. X. and Yoo, C. D. One-step flow q-learning: Addressing the diffusion policy bottleneck in offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.
- Park, S., Li, Q., and Levine, S. Is value learning really the main bottleneck in offline reinforcement learning? In *Neural Information Processing Systems (NeurIPS)*, 2024.
- Park, S., Frans, K., Eysenbach, B., and Levine, S. Ogbench: Benchmarking offline goal-conditioned reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Park, S., Li, Q., and Levine, S. Flow q-learning: Addressing the diffusion policy bottleneck in offline reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2025b.
- Peters, J. and Schaal, S. Reinforcement learning by reward-weighted regression for operational space control. In *International Conference on Machine Learning (ICML)*, 2007.
- Prudencio, R. F., Máximo, M. R. O. A., and Colombini, E. L. A survey on offline reinforcement learning: Taxonomy, review, and open problems. *IEEE Transactions on Neural Networks and Learning Systems*, 35(8):10237–10257, 2024.
- Qing, Y., Liu, S., Cong, J., Chen, K., Zhou, Y., and Song, M. A2po: Towards effective offline reinforcement learning from an advantage-aware perspective. In *Neural Information Processing Systems (NeurIPS)*, 2024.

- Ren, A. Z., Lidard, J., Ankile, L. L., Simeonov, A., Agrawal, P., Majumdar, A., Burchfiel, B., Dai, H., and Simchowitz, M. Diffusion policy optimization. In *International Conference on Learning Representations (ICLR)*, 2025.
- Shin, Y., Chae, J., Park, J., and Sung, Y. Flow matching with injected noise for offline-to-online reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.
- Tarasov, D., Kurenkov, V., Nikulin, A., and Kolesnikov, S. Revisiting the minimalist approach to offline reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2023a.
- Tarasov, D., Nikulin, A., Akimov, D., Kurenkov, V., and Kolesnikov, S. Corl: Research-oriented deep offline reinforcement learning library. In *Neural Information Processing Systems (NeurIPS)*, 2023b.
- Tiofack, F. N., Hellard, T. L., Schramm, F., Perrin-Gilbert, N., and Carpentier, J. Guided flow policy: Learning from high-value actions in offline reinforcement learning. *arXiv preprint arXiv:2512.03973*, 2025.
- Venkatraman, S., Khaitan, S., Akella, R. T., Dolan, J., Schneider, J., and Berseth, G. Reasoning with latent diffusion in offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2024.
- Wang, J., Li, L., Wei, W., Zhang, Y., and Yang, X. Dynamic uncertainty estimation for offline reinforcement learning. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2025.
- Wang, Z., Hunt, J. J., and Zhou, M. Diffusion policies as an expressive policy class for offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023.
- Wang, Z., Li, D., Chen, Y., Shi, Y., Bai, L., Yu, T., and Fu, Y. One-step generative policies with q-learning: A reformulation of meanflow. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2026.
- Wu, Y., Tucker, G., and Nachum, O. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361*, 2019.
- Yu, T., Kumar, A., Rafailov, R., Rajeswaran, A., Levine, S., and Finn, C. Combo: Conservative offline model-based policy optimization. In *Neural Information Processing Systems (NeurIPS)*, 2021.
- Zhang, S., Zhang, W., and Gu, Q. Energy-weighted flow matching for offline reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025a.
- Zhang, T., Yu, C., Su, S., and Wang, Y. Reinflow: Fine-tuning flow matching policy with online reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2025b.
- Zhang, Y., Yu, S., Zhang, T., Guang, M., Hui, H., Long, K., Wang, Y., Yu, C., and Ding, W. Sac flow: Sample-efficient reinforcement learning of flow-based policies via velocity-reparameterized sequential modeling. *arXiv preprint arXiv:2509.25756*, 2025c.
- Zhao, A., Lin, M., Li, Y., jin Liu, Y., and Huang, G. A mixture of surprises for unsupervised reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2022.
- Zhao, Z., Guo, H., Chen, S., Xu, K., Jiang, B., Zhu, Y., and Zhao, D. Empowering multi-robot cooperation via sequential world models. In *International Conference on Learning Representations (ICLR)*, 2026.
- Zhou, W., Bajracharya, S., and Held, D. Plas: Latent action space for offline reinforcement learning. In *Conference on Robot Learning (CoRL)*, 2020.

A. Derivations

A.1. Gradient equivalence for RL loss

This section provides a formal derivation showing that the gradient of the quadratic loss in Equation 14 is equivalent to the gradient of the inner product objective in Equation 15. Starting from the loss function $\mathcal{L}_k(\theta)$ defined in Equation 14:

$$\mathcal{L}_k(\theta) = \frac{1}{2} \|v_\theta(s, a_k, t_k) - \text{sg}[\tilde{v}_k]\|^2 \quad (22)$$

where $\text{sg}[\cdot]$ is the stop-gradient operator. Let v_θ denote $v_\theta(s, a_k, t_k)$ for brevity. The gradient of this scalar loss with respect to the parameters θ is computed as:

$$\nabla_\theta \mathcal{L}_k(\theta) = (v_\theta - \text{sg}[\tilde{v}_k])^\top \nabla_\theta v_\theta. \quad (23)$$

By the definition of the stop-gradient operator, $\text{sg}[\tilde{v}_k]$ is treated as a constant vector during differentiation. According to Equation 13, we have $\tilde{v}_k = v_\theta + \Delta t \cdot g$. When evaluating the gradient at the current parameter state θ , the value inside the stop-gradient operator matches the current output of the network. We can denote this fixed value as $v_{\text{fixed}} = v_\theta|_\theta$. Thus:

$$\nabla_\theta \mathcal{L}_k(\theta) = (v_\theta - (v_{\text{fixed}} + \Delta t \cdot g))^\top \nabla_\theta v_\theta. \quad (24)$$

Expanding the terms, we obtain:

$$\nabla_\theta \mathcal{L}_k(\theta) = (v_\theta - v_{\text{fixed}})^\top \nabla_\theta v_\theta - (\Delta t \cdot g)^\top \nabla_\theta v_\theta. \quad (25)$$

At the point of evaluation, the current velocity v_θ is numerically equal to the fixed velocity v_{fixed} , which implies that the term $(v_\theta - v_{\text{fixed}})$ becomes zero. Consequently, the first term vanishes:

$$\nabla_\theta \mathcal{L}_k(\theta) = -\Delta t \cdot g^\top \nabla_\theta v_\theta. \quad (26)$$

Since we treat g as a gradient-clipped constant here, both g and Δt are independent of θ . Therefore, we can rewrite the expression as the gradient of an inner product:

$$\nabla_\theta \mathcal{L}_k(\theta) = \nabla_\theta (-\Delta t \cdot g^\top v_\theta(s, a_k, t_k)). \quad (27)$$

This completes the proof that the quadratic loss with a stop-gradient is equivalent to the linear projection-based optimization in terms of gradient descent.

A.2. Asymptotic consistency of the temporally weighted objective

In this section, we provide a formal proof of the asymptotic consistency for the proposed objective in Equation 16. We demonstrate that, for a Lipschitz-continuous v_θ , by employing the temporal weighting $w_k = 1/(K - k)$, the cumulative change of the terminal state a_K asymptotically aligns with the guidance direction g as the number of discrete steps $K \rightarrow \infty$. Our proof first analyzes the contribution of the change at time step k to the terminal state under the loss defined in Equation 16. We then examine how the aggregation of these contributions across all time steps affects the alignment between the resulting terminal-state update direction and the desired direction g .

Variational perturbation in velocity space. We analyze the optimization through the lens of functional gradient descent in the velocity space. Given the objective $\mathcal{L}_{\text{RL}} = -\sum_{k=0}^{K-1} w_k \Delta t \cdot g^\top v_k$, the steepest descent direction with respect to the velocity field at step k is defined by the functional derivative:

$$-\frac{\delta \mathcal{L}_{\text{RL}}}{\delta v_k} = w_k \Delta t \cdot g. \quad (28)$$

A gradient update with an effective learning rate $\eta > 0$ induces a local velocity perturbation $\delta v_k = \eta w_k \Delta t \cdot g$.

Propagation through discrete dynamics. To evaluate the impact of a local perturbation δv_k on the terminal state a_K , we examine the perturbation propagation through the discrete-time dynamical system. The state at step $k + 1$ is updated as $a_{k+1} = a_k + \Delta t \cdot v_\theta(a_k, t_k)$. The local velocity change δv_k induces an immediate displacement:

$$\delta a_{k+1} = \Delta t \cdot \delta v_k = \eta w_k (\Delta t)^2 \cdot g. \quad (29)$$

Consider the propagation of this displacement to the subsequent step a_{k+2} . Applying a first-order Taylor expansion to the evolution equation:

$$a_{k+2} + \delta a_{k+2} = (a_{k+1} + \delta a_{k+1}) + \Delta t \cdot v_\theta(s, a_{k+1} + \delta a_{k+1}, t_{k+1}), \quad (30)$$

$$\delta a_{k+2} \approx \delta a_{k+1} + \Delta t \frac{\partial v_\theta}{\partial a} \Big|_{a_{k+1}} \delta a_{k+1} = (I + \Delta t \cdot J_{k+1}) \delta a_{k+1}, \quad (31)$$

where J_{k+1} denotes the Jacobian matrix of the velocity field at step $k+1$. By recursive application, the terminal displacement $\Delta a_K^{(k)}$ originating from step k is given by:

$$\Delta a_K^{(k)} \triangleq \left(\prod_{j=k+1}^{K-1} (I + \Delta t \cdot J_j) \right) \delta a_{k+1} = \Phi_{k \rightarrow K} \delta a_{k+1}, \quad (32)$$

where $\Phi_{k \rightarrow K}$ is the discrete propagator characterizing the cumulative influence of the system dynamics on the initial perturbation.

Effective propagator and linearization. Assuming the velocity field $v_\theta(s, a, t)$ is Lipschitz continuous in a , the sequence of Jacobians is bounded, (i.e., $\|J_j\| \leq L$). For a sufficiently large K (small $\Delta t = 1/K$), the propagator can be linearized as:

$$\Phi_{k \rightarrow K} = I + \Delta t \sum_{j=k+1}^{K-1} J_j + \mathcal{O}(\Delta t^2). \quad (33)$$

We define the effective average Jacobian over the remaining path as $\bar{J}_i = \frac{1}{i} \sum_{j=k+1}^{K-1} J_j$, where $i = K - 1 - k$ is the number of remaining steps. The propagator simplifies to:

$$\Phi_{k \rightarrow K} \approx I + \frac{i}{K} \bar{J}_i. \quad (34)$$

This term represents the first-order approximation of the dynamical drift that potentially rotates the intended update direction g .

Asymptotic directional alignment. The total displacement Δa_{total} is the summation of contributions from all steps $k \in \{0, \dots, K-1\}$. Substituting the specific weighting $w_k = 1/(K-k) = 1/(i+1)$:

$$\Delta a_{\text{total}} = \sum_{i=0}^{K-1} \Delta a_K^{(i)} = \frac{\eta}{K^2} \sum_{i=0}^{K-1} \left[\frac{1}{i+1} g + \frac{i}{K(i+1)} \bar{J}_i g \right]. \quad (35)$$

We analyze the two components of this summation as $K \rightarrow \infty$:

i) The Guidance Term: The first component scales with the harmonic series $\sum_{i=0}^{K-1} \frac{1}{i+1}$, which diverges logarithmically:

$$\left(\sum_{i=0}^{K-1} \frac{1}{i+1} \right) g \rightarrow (\ln K) g. \quad (36)$$

ii) The Dynamical Drift Term: The second component represents the accumulated bias from the Jacobians. Under the Lipschitz assumption ($\|\bar{J}_i\| \leq L$):

$$\left\| \sum_{i=0}^{K-1} \frac{i}{K(i+1)} \bar{J}_i g \right\| \leq \frac{L\|g\|}{K} \sum_{i=0}^{K-1} 1 = L\|g\|. \quad (37)$$

This term is bounded by a constant independent of K . Combining these results, the terminal displacement follows the asymptotic form :

$$\Delta a_{\text{total}} \propto (\ln K) g + \mathcal{O}(L) g. \quad (38)$$

As $K \rightarrow \infty$, the logarithmic term dominates the constant drift. Consequently, the direction of the total improvement aligns with the guidance signal:

$$\lim_{K \rightarrow \infty} \text{cosine similarity}(\Delta a_{\text{total}}, g) = 1. \quad (39)$$

This concludes the proof that, under the Lipschitz continuity assumption on v_θ , the temporal weighting $w_k = 1/(K-k)$ scheme ensures first-order consistency with the terminal objective.

B. Limitations

While DFQL demonstrates superior performance and stability, we identify several limitations that offer promising directions for future research.

Inference latency vs. expressivity. One primary limitation of DFQL is the inherent trade-off between representation capability and computational efficiency. By preserving the full iterative structure of flow matching models, DFQL maintains higher expressivity compared to distillation-based approaches. However, the multi-step sampling process results in increased inference latency. Although our proposed action-chunking variant, QC-DFQL, partially can alleviate the execution burden by outputting multiple actions in a single inference pass, thereby reducing the required frequency of policy network execution, a more fundamental solution may lie in combining techniques such as Mean Flow Matching (Geng et al., 2025) that directly enhance inference speed at the flow network architecture level. Nevertheless, Mean Flow Matching often imposes more stringent requirements on training stability and data volume compared to standard Conditional Flow Matching. The variance introduced by the critic during RL further complicates this optimization, making the stable joint training of Mean-Flow-Matching-based policies an open and valuable challenge.

Hyperparameter α in policy extraction. Like many offline RL frameworks following the behavior-constrained paradigm (Tarasov et al., 2023b; Park et al., 2025b), the performance of DFQL is sensitive to the hyperparameter α , which balances the BC ($\mathcal{L}_{BC}$) and RL ($\mathcal{L}_{RL}$) components. While some recent studies have explored adaptive strategies using dual optimization or reward-scaling heuristics, effectively automating this trade-off for generative policies remains a significant challenge. Existing mechanisms frequently suffer from high sensitivity to initial hyperparameter settings or rely on metrics such as the Kullback–Leibler divergence, which are computationally expensive to evaluate for flow matching models due to the need for ODE integration. Therefore, developing a robust, initialization-insensitive, and architecture-agnostic automated tuning mechanism for α represents a critical and ongoing research direction in generative policy extraction.

C. Implementation Details and Ablation Experiments

In this section, we provide the comprehensive implementation specifics of DFQL along with ablation studies that justify our core design choices.

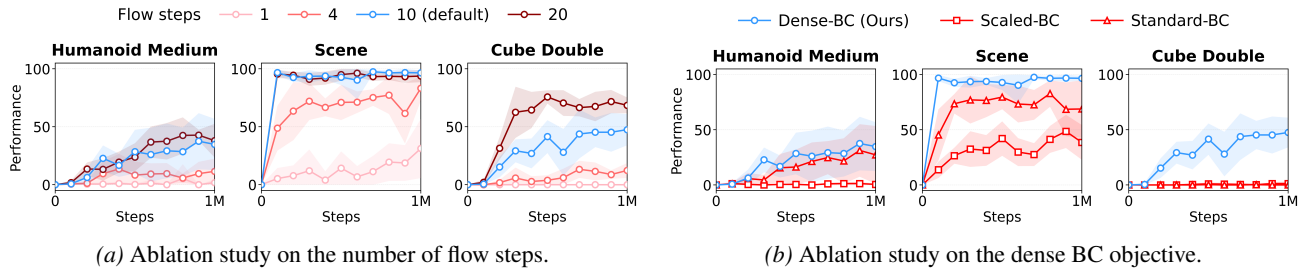


Figure 5. Ablation studies on key components of our framework.

Discretization strategy and selection of K . The generation process of our flow-based policy is implemented using Euler discretization steps without incorporating sinusoidal temporal embeddings. Theoretically, increasing the number of discrete ODE steps K minimizes approximation bias, though at the expense of training and inference efficiency. As evidenced in Figure 5a, while performance generally scales with K , the necessity for higher values varies by task. For the majority of environments, such as “humanoidmaze-medium-singletask” and “scene-singletask”, $K = 10$ is already sufficient. Although complex scenarios like “cube-double-singletask” still benefit from further increases, we prioritize the balance between performance and computational overhead. Consequently, aligning with the configuration in FQL, we adopt $K = 10$ for all other experiments and recommend it as a robust initial setting for users.

Implementation and ablation of the dense BC objective. Following the foundations in Section 2, our framework utilizes a flow-matching objective based on linear probability paths. To maintain structural symmetry and ensure consistent supervision density between the RL objective and the BC constraint, we adopt the dense BC objective introduced in Section 3.3. To evaluate the impact of this design (Equation 20), we conduct ablation studies across three representative tasks, comparing it against the conventional uniform random sampling scheme (results illustrated in Figure 5b). To distinguish the benefits of increased supervision density from a simple increase in constraint magnitude, we introduce two specific baselines: (i)

standard uniform temporal sampling, and (ii) a linearly scaled version where the standard BC component ($\mathcal{L}_{BC}$) is multiplied by the number of discrete steps K to simulate higher intensity. Our findings reveal that the proposed dense BC objective achieves the best performance, followed by the standard approach. Notably, the linear scaling of BC intensity yields the poorest results, as it disrupts the optimal balance between BC and RL signals. This ablation highlights the utility of the dense BC design, which further bolsters our method’s performance and ensures a robust training signal alongside our core framework.

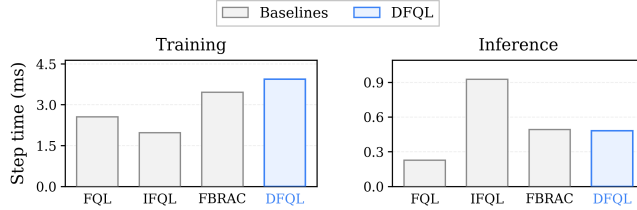


Figure 6. **Run time comparison.** The run times are measured on the same machine and are averaged over 8 seeds.

Computational cost. We profile the wall-clock cost of major flow-based baselines on the representative cube-double-play-singletask-v0 environment under the same machine, codebase, and seed setting. As shown in Figure 6, DFQL has higher inference latency than the one-step FQL baseline, while being comparable to FBRAC and faster than IFQL. This trend is consistent with the computational structures of these methods: FQL performs one-step action generation, FBRAC and DFQL both preserve an iterative flow-based sampling process, and IFQL additionally requires selecting from multiple sampled candidates. During training, DFQL incurs a higher per-update cost. This is mainly because DFQL applies RL guidance to every flow step, and the dense BC objective introduces additional step-wise supervision. Therefore, the additional runtime cost reflects the expected computational trade-off of preserving the multi-step flow structure instead of distilling it into a one-step policy.

Value learning and target aggregation. We maintain an ensemble of two Q-networks to enhance optimization stability. In both the policy improvement phase and the TD target calculation, the Q-values are aggregated using either the mean or the minimum, depending on the specific task environment. We also utilize target Q networks whose parameters are updated via exponential moving average. By default, the mean value is employed for both the actor’s objective and the TD target. However, for Adroit and OGBench Antmaze tasks, we adopt the minimum of the two Q-values (Clipped Double Q-learning) to leverage its pessimistic advantages in these domains. These value learning configurations are strictly aligned with the settings used in FQL to ensure a fair and consistent comparison.

Network architectures. In alignment with the standard settings in FQL, we utilize multi-layer perceptrons with a hidden layer structure of [512, 512, 512, 512] for all neural components and incorporate layer normalization in the value networks. However, we found a rare forward-pass numerical overflow when the learned critic provides unreliable velocity targets for noise samples in extremely low-probability regions, and the actor’s velocity field is iteratively integrated for multiple steps. This issue arises during multi-step sampling/integration rather than from backpropagated gradient explosion. To avoid undefined rollouts, we apply a simple numerical guardrail by bounding the actor output with $5 \times \tanh$ only in DFQL. Importantly, this modification is not part of our algorithmic contribution; it only ensures stable evaluation rollouts. As shown in Table 3, applying this bounded activation to the FQL baseline actually leads to performance degradation. Given that FQL serves as our strongest competitive baseline, we choose to maintain its original standard architecture, as well as those of other baselines, to ensure they are evaluated under their respective optimal configurations for a fair comparison.

Online fine-tuning. For the transition from offline to online RL, we follow a simple protocol of appending online interaction data to the existing offline buffer. The training objectives for both the actor and critic remain identical to those used in the offline phase, as described in Algorithm 1.

Image processing. For tasks involving pixel-based observations, we leverage a compact version of the IMPALA encoder. Specifically, we apply random-shift augmentations with a probability of 0.5 to improve generalization. Furthermore, to capture temporal dynamics effectively, we utilize frame stacking with three consecutive images as input.

Training and evaluation. The training duration is set to 1M gradient steps for state-based OGBench environments, while a reduced schedule of 500K steps is applied to D4RL and pixel-based OGBench tasks. Each agent undergoes evaluation every 100K steps across 50 episodes. Following the established OGBench evaluation framework (Park et al., 2025a), we

report the average success rate of the last three epochs; meanwhile, D4RL results reflect the performance at the final epoch. For offline-to-online transitions, we document performance specifically at the 1M (offline completion) and 2M (online completion) marks.

Coefficient α . The most critical hyperparameter in our unified objective is the coefficient α , which balances the imitation and reinforcement signals. We conduct a grid search for α over $\{0.0033, 0.01, 0.02, 0.033, 0.1, 0.2\}$ for Adroit tasks and $\{0.2, 0.33, 1, 2, 3.3, 10, 20, 33\}$ for all other environments, selecting the optimal value for each specific task. The lower range for Adroit is necessitated by the significantly larger scale of its return values compared to other benchmarks. In practice, we recommend selecting alpha through a two-stage procedure. One may first perform a coarse search to identify the appropriate order of magnitude, and then refine alpha within a small local range.

Baselines. We adopt the same baseline configurations as those utilized in FQL (Park et al., 2025b), ensuring full methodological alignment in both implementation and evaluation protocols. Thanks to the exhaustive and detailed experimental analysis provided by the authors of FQL, and given that our experimental setup is virtually identical to theirs, we are able to directly leverage many of their reported benchmark results for our comparative analysis to facilitate a fair assessment. All baselines are executed using the official FQL codebase without modification, with hyperparameters strictly retained as reported in the original study.

Hyperparameters. The complete set of hyperparameters is presented in Tables 6 to 8.

Table 3. Reproduction check and bounded-activation ablation for the FQL baseline. We compare the FQL results reported in the original paper, our reproduced results using the official FQL codebase, and a variant that adds the bounded activation function ($5 \times \tanh$) to the policy output. The reproduced results closely match the reported FQL performance, while adding the bounded activation leads to a slight decrease in overall average performance. Therefore, for all formal comparisons, we keep the standard FQL architecture without the additional bounded activation to avoid penalizing the baseline.

Task	FQL reported	FQL reproduced	FQL + bounded tanh activation
antmaze-large-navigate-singletask-task1-v0	80 $\pm$ 8	79 $\pm$ 11	73 $\pm$ 11
antmaze-large-navigate-singletask-task2-v0	57 $\pm$ 10	61 $\pm$ 11	52 $\pm$ 12
antmaze-large-navigate-singletask-task3-v0	93 $\pm$ 3	95 $\pm$ 4	93 $\pm$ 4
antmaze-large-navigate-singletask-task4-v0	80 $\pm$ 4	85 $\pm$ 7	79 $\pm$ 10
antmaze-large-navigate-singletask-task5-v0	83 $\pm$ 4	84 $\pm$ 7	80 $\pm$ 8
antmaze-giant-navigate-singletask-task1-v0	4 $\pm$ 5	10 $\pm$ 9	6 $\pm$ 9
antmaze-giant-navigate-singletask-task2-v0	9 $\pm$ 7	7 $\pm$ 11	7 $\pm$ 12
antmaze-giant-navigate-singletask-task3-v0	0 $\pm$ 1	0 $\pm$ 1	0 $\pm$ 0
antmaze-giant-navigate-singletask-task4-v0	14 $\pm$ 23	26 $\pm$ 28	4 $\pm$ 13
antmaze-giant-navigate-singletask-task5-v0	16 $\pm$ 28	13 $\pm$ 26	11 $\pm$ 22
humanoidmaze-medium-navigate-singletask-task1-v0	19 $\pm$ 12	29 $\pm$ 21	23 $\pm$ 22
humanoidmaze-medium-navigate-singletask-task2-v0	94 $\pm$ 3	96 $\pm$ 6	92 $\pm$ 13
humanoidmaze-medium-navigate-singletask-task3-v0	74 $\pm$ 18	59 $\pm$ 40	53 $\pm$ 37
humanoidmaze-medium-navigate-singletask-task4-v0	3 $\pm$ 4	0 $\pm$ 2	1 $\pm$ 3
humanoidmaze-medium-navigate-singletask-task5-v0	97 $\pm$ 2	94 $\pm$ 15	99 $\pm$ 1
humanoidmaze-large-navigate-singletask-task1-v0	7 $\pm$ 6	3 $\pm$ 3	4 $\pm$ 6
humanoidmaze-large-navigate-singletask-task2-v0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0
humanoidmaze-large-navigate-singletask-task3-v0	11 $\pm$ 7	13 $\pm$ 8	11 $\pm$ 11
humanoidmaze-large-navigate-singletask-task4-v0	2 $\pm$ 3	1 $\pm$ 2	2 $\pm$ 4
humanoidmaze-large-navigate-singletask-task5-v0	1 $\pm$ 3	1 $\pm$ 3	3 $\pm$ 6
antsoccer-arena-navigate-singletask-task1-v0	77 $\pm$ 4	79 $\pm$ 7	80 $\pm$ 12
antsoccer-arena-navigate-singletask-task2-v0	88 $\pm$ 3	88 $\pm$ 6	85 $\pm$ 7
antsoccer-arena-navigate-singletask-task3-v0	61 $\pm$ 6	52 $\pm$ 10	50 $\pm$ 10
antsoccer-arena-navigate-singletask-task4-v0	39 $\pm$ 6	39 $\pm$ 10	36 $\pm$ 10
antsoccer-arena-navigate-singletask-task5-v0	36 $\pm$ 9	32 $\pm$ 14	29 $\pm$ 15
cube-single-play-singletask-task1-v0	97 $\pm$ 2	97 $\pm$ 4	97 $\pm$ 3
cube-single-play-singletask-task2-v0	97 $\pm$ 2	97 $\pm$ 4	98 $\pm$ 3
cube-single-play-singletask-task3-v0	98 $\pm$ 2	98 $\pm$ 2	98 $\pm$ 3
cube-single-play-singletask-task4-v0	94 $\pm$ 3	95 $\pm$ 3	97 $\pm$ 2
cube-single-play-singletask-task5-v0	93 $\pm$ 3	94 $\pm$ 4	93 $\pm$ 6
cube-double-play-singletask-task1-v0	61 $\pm$ 9	61 $\pm$ 9	56 $\pm$ 11
cube-double-play-singletask-task2-v0	36 $\pm$ 6	34 $\pm$ 8	36 $\pm$ 10
cube-double-play-singletask-task3-v0	22 $\pm$ 5	20 $\pm$ 7	24 $\pm$ 12
cube-double-play-singletask-task4-v0	5 $\pm$ 2	5 $\pm$ 3	5 $\pm$ 2
cube-double-play-singletask-task5-v0	19 $\pm$ 10	15 $\pm$ 10	16 $\pm$ 10
scene-play-singletask-task1-v0	100 $\pm$ 0	100 $\pm$ 0	100 $\pm$ 1
scene-play-singletask-task2-v0	76 $\pm$ 9	83 $\pm$ 13	79 $\pm$ 14
scene-play-singletask-task3-v0	98 $\pm$ 1	96 $\pm$ 3	98 $\pm$ 3
scene-play-singletask-task4-v0	5 $\pm$ 1	10 $\pm$ 10	9 $\pm$ 10
scene-play-singletask-task5-v0	0 $\pm$ 0	0 $\pm$ 0	0 $\pm$ 0
puzzle-3x3-play-singletask-task1-v0	90 $\pm$ 4	89 $\pm$ 8	93 $\pm$ 5
puzzle-3x3-play-singletask-task2-v0	16 $\pm$ 5	13 $\pm$ 7	12 $\pm$ 5
puzzle-3x3-play-singletask-task3-v0	10 $\pm$ 3	9 $\pm$ 4	10 $\pm$ 7
puzzle-3x3-play-singletask-task4-v0	16 $\pm$ 5	11 $\pm$ 6	13 $\pm$ 7
puzzle-3x3-play-singletask-task5-v0	16 $\pm$ 3	18 $\pm$ 7	22 $\pm$ 11
puzzle-4x4-play-singletask-task1-v0	34 $\pm$ 8	30 $\pm$ 8	32 $\pm$ 8
puzzle-4x4-play-singletask-task2-v0	16 $\pm$ 5	14 $\pm$ 5	12 $\pm$ 6
puzzle-4x4-play-singletask-task3-v0	18 $\pm$ 5	19 $\pm$ 9	22 $\pm$ 5
puzzle-4x4-play-singletask-task4-v0	11 $\pm$ 3	10 $\pm$ 5	9 $\pm$ 5
puzzle-4x4-play-singletask-task5-v0	7 $\pm$ 3	6 $\pm$ 4	7 $\pm$ 4
Overall Mean	44	43	42

D. QC-DFQL

In this section, we provide a comprehensive description of QC-DFQL, a modular extension of our framework that integrates DFQL with the Q-chunking (QC) recipe. By extending flow-matching policies to a temporally extended action space, QC-DFQL leverages the stability of our proposed optimization objective while benefiting from the improved exploration and credit assignment provided by action chunking.

D.1. Theoretical Background of Action Chunking

The core algorithmic modification of QC lies in the fundamental shift of the action dimension, which offers two-fold advantages for offline/offline-to-online RL in challenging environments. Instead of predicting a single action at each timestep,

the agent learns to output an ‘‘action chunk’’ $a_{n:n+h} = [a_n, a_{n+1}, \dots, a_{n+h-1}] \in \mathbb{R}^{dh}$, which is executed open-loop for h steps. The advantages of QC are mainly reflected in the following two aspects:

(i) Temporally coherent exploration. By operating on action sequences, the framework facilitates exploration through temporally consistent behaviors. Unlike standard Markovian policies that may produce jittery or inconsistent movements, action chunking allows the agent to leverage structured behaviors (e.g., reaching or grasping) inherent in the offline data. This consistency significantly improves state coverage and sample efficiency during the online fine-tuning phase.

(ii) Unbiased multi-step backups. The ‘‘chunked’’ formulation enables stable and accelerated value backups. While traditional n -step returns often suffer from off-policy bias when the behavior policy differs from the current policy, a critic trained on action chunks maintains an unbiased value estimate. This is because the h -step Q-function takes as input the exact sequence of actions used to collect the corresponding h -step rewards, allowing the value signal to propagate backward h times faster without the typical bias found in uncorrected multi-step TD methods.

D.2. Orthogonality and Synergistic Potential

A key motivation for developing QC-DFQL is the observation that QC and DFQL target distinct stages of the RL pipeline. Q-chunking primarily focuses on the representation and credit assignment stage, optimizing the structure of the action space and the efficiency of value propagation. It provides a recipe for how the agent perceives and interacts with time and action, but it does not inherently dictate how the underlying policy should be optimized.

In contrast, DFQL addresses the policy extraction and optimization stage. Its primary contribution is providing a stable optimization pathway for iterative flow-matching models by mitigating the gradient explosion problem inherent in BPTT. The original Q-chunking framework integrates the chunking design with FQL to produce a high-performance and efficient variant QC-FQL (Li et al., 2025). By replacing the FQL component with our proposed DFQL, we inherit the structural benefits of action chunking while ensuring robust optimization stability in the high-dimensional action chunk space $\mathbb{R}^{dh}$ without sacrificing the model’s iterative expressive capacity. This structural synergy allows DFQL to serve as a stable backbone for the temporally extended policies required by QC, leading to superior performance in complex, sparse-reward manipulation tasks.

D.3. Specific Training Formulas and Implementation Details

Following the established framework for temporally extended action spaces, we extract transitions consisting of consecutive state-action sequences from the dataset. Let $(s_n, \mathbf{a}_n, s_{n+h}, r_n^h) \sim \mathcal{D}$ denote a sampled transition, where $\mathbf{a}_n = [a_n, \dots, a_{n+h-1}] \in \mathbb{R}^{dh}$ represents the action chunk and $r_n^h = \sum_{n'=0}^{h-1} \gamma^{n'} r_{n+n'}$ is the cumulative discounted reward. The optimization objectives for QC-DFQL are defined as follows:

The value function is trained to minimize the h -step TD error by considering the entire action sequence:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{\substack{(s_n, \mathbf{a}_n, s_{n+h}, r_n^h) \sim \mathcal{D} \\ \mathbf{a}_{n+h} \sim \pi_\theta(\cdot | s_{n+h})}} \left[(Q_\phi(s_n, \mathbf{a}_n) - r_n^h - \gamma^h Q_{\bar{\phi}}(s_{n+h}, \mathbf{a}_{n+h}))^2 \right]. \quad (40)$$

For the policy extraction stage, we optimize the velocity field v_θ by injecting gradients from the ‘‘chunked’’ critic. The RL guidance objective is lifted to the sequence space $\mathbb{R}^{dh}$ as:

$$\mathcal{L}_{\text{RL}}(\theta) = -\mathbb{E}_{\substack{(s_n, \mathbf{a}_n, s_{n+h}, r_n^h) \sim \mathcal{D} \\ \mathbf{a}_0^{\text{RL}} \sim \mathcal{N}(0, I_{dh})}} \left[\sum_{k=0}^{K-1} (w_k \Delta t \cdot g^\top v_\theta(s_n, \mathbf{a}_k^{\text{RL}}, t_k)) \right], \quad (41)$$

where $g = \nabla_{\mathbf{a}_K^{\text{RL}}} Q_\phi(s_n, \mathbf{a}_K^{\text{RL}})$ and $\mathbf{a}_{k+1}^{\text{RL}} = \mathbf{a}_k^{\text{RL}} + \Delta t \cdot v_\theta(s_n, \mathbf{a}_k^{\text{RL}}, t_k)$. Simultaneously, the QC-DFQL BC loss is

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{\substack{(s_n, \mathbf{a}_n, s_{n+h}, r_n^h) \sim \mathcal{D} \\ \mathbf{a}_0^{\text{BC}} \sim \mathcal{N}(0, I_{dh})}} \left[\sum_{k=0}^{K-1} \|v_\theta(s_n, (1-t_k)\mathbf{a}_0^{\text{BC}} + t_k \mathbf{a}_n, t_k) - (\mathbf{a}_n - \mathbf{a}_0^{\text{BC}})\|_2^2 \right]. \quad (42)$$

By utilizing these objectives, QC-DFQL maintains optimization stability while leveraging the exploration benefits of temporally coherent actions. For the implementation, the hyperparameter α in QC-DFQL is set to 1 across all tasks, while all other shared configurations remain identical to those of DFQL. Regarding the chunk size, we adopt the setting $h = 5$ to maintain consistency with QC-FQL, thereby ensuring a fair comparison. The complete experimental comparison is shown in Figure 7.

E. Full Results

The complete per-task offline RL results are provided in Table 5, while the full offline-to-online RL results are shown in Table 4. Moreover, we present the comprehensive task-by-task training curves for the QC-DFQL and QC-FQL integration study in Figure 7. We report the standard deviation using the “ $\pm$ ” notation in the tables and indicate the 95% bootstrap confidence interval with shaded regions in the figures. Following the convention of OGBench (Park et al., 2025a), we highlight values in bold if they are at or above 95% of the best performance.

Table 4. **Full Offline-to-online RL results.** Results are reported as the mean over 8 seeds.

Task	IQL	ReBRAC	Cal-QL	RLPD	IFQL	FQL	DFQL
humanoidmaze-medium-navigate	$21 \pm 13 \rightarrow 16 \pm 8$	$16 \pm 20 \rightarrow 1 \pm 1$	$0 \pm 0 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow 8 \pm 10$	$56 \pm 35 \rightarrow \mathbf{82} \pm 20$	$12 \pm 7 \rightarrow 22 \pm 12$	$33 \pm 26 \rightarrow 77 \pm 30$
antsoccer-arena-navigate	$2 \pm 1 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow 0 \pm 0$	$26 \pm 15 \rightarrow 39 \pm 10$	$28 \pm 8 \rightarrow \mathbf{86} \pm 5$	$47 \pm 6 \rightarrow \mathbf{89} \pm 6$
cube-double-play	$0 \pm 1 \rightarrow 0 \pm 0$	$6 \pm 5 \rightarrow 28 \pm 28$	$0 \pm 0 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow 0 \pm 0$	$12 \pm 9 \rightarrow 40 \pm 5$	$40 \pm 11 \rightarrow \mathbf{92} \pm 3$	$49 \pm 23 \rightarrow \mathbf{91} \pm 7$
scene-play	$14 \pm 11 \rightarrow 10 \pm 9$	$55 \pm 10 \rightarrow \mathbf{100} \pm 0$	$1 \pm 2 \rightarrow 50 \pm 53$	$0 \pm 0 \rightarrow \mathbf{100} \pm 0$	$0 \pm 1 \rightarrow 60 \pm 39$	$82 \pm 11 \rightarrow \mathbf{100} \pm 1$	$97 \pm 5 \rightarrow \mathbf{100} \pm 0$
puzzle-4x4-play	$5 \pm 2 \rightarrow 1 \pm 1$	$8 \pm 4 \rightarrow 14 \pm 35$	$0 \pm 0 \rightarrow 0 \pm 0$	$0 \pm 0 \rightarrow \mathbf{100} \pm 1$	$23 \pm 6 \rightarrow 19 \pm 33$	$38 \pm 3 \rightarrow 38 \pm 52$	$14 \pm 3 \rightarrow 87 \pm 35$
antmaze-umaze-v2	$77 \rightarrow \mathbf{96}$	$98 \rightarrow 75$	$77 \rightarrow \mathbf{100}$	$0 \pm 0 \rightarrow \mathbf{98} \pm 3$	$94 \pm 5 \rightarrow \mathbf{96} \pm 2$	$97 \pm 2 \rightarrow \mathbf{99} \pm 1$	$95 \pm 3 \rightarrow \mathbf{100} \pm 0$
antmaze-umaze-diverse-v2	$60 \rightarrow 64$	$74 \rightarrow \mathbf{98}$	$32 \rightarrow \mathbf{98}$	$0 \pm 0 \rightarrow 94 \pm 5$	$69 \pm 20 \rightarrow 93 \pm 5$	$79 \pm 16 \rightarrow \mathbf{100} \pm 1$	$90 \pm 4 \rightarrow \mathbf{99} \pm 2$
antmaze-medium-play-v2	$72 \rightarrow 90$	$88 \rightarrow \mathbf{98}$	$72 \rightarrow \mathbf{99}$	$0 \pm 0 \rightarrow \mathbf{98} \pm 2$	$52 \pm 19 \rightarrow 93 \pm 2$	$77 \pm 7 \rightarrow \mathbf{97} \pm 2$	$84 \pm 5 \rightarrow \mathbf{96} \pm 4$
antmaze-medium-diverse-v2	$64 \rightarrow 92$	$85 \rightarrow \mathbf{99}$	$62 \rightarrow \mathbf{98}$	$0 \pm 0 \rightarrow \mathbf{97} \pm 2$	$44 \pm 26 \rightarrow 89 \pm 4$	$55 \pm 19 \rightarrow \mathbf{97} \pm 3$	$79 \pm 8 \rightarrow \mathbf{98} \pm 2$
antmaze-large-play-v2	$38 \rightarrow 64$	$68 \rightarrow 32$	$32 \rightarrow \mathbf{97}$	$0 \pm 0 \rightarrow \mathbf{93} \pm 5$	$64 \pm 14 \rightarrow 80 \pm 5$	$66 \pm 40 \rightarrow 84 \pm 30$	$75 \pm 5 \rightarrow 92 \pm 3$
antmaze-large-diverse-v2	$27 \rightarrow 64$	$67 \rightarrow 72$	$44 \rightarrow \mathbf{92}$	$0 \pm 0 \rightarrow \mathbf{94} \pm 3$	$69 \pm 6 \rightarrow 86 \pm 5$	$75 \pm 14 \rightarrow \mathbf{94} \pm 3$	$75 \pm 7 \rightarrow \mathbf{93} \pm 4$
pen-cloned-v1	$84 \rightarrow 102$	$74 \rightarrow 138$	$-3 \rightarrow -3$	$3 \pm 2 \rightarrow 120 \pm 10$	$77 \pm 7 \rightarrow 107 \pm 10$	$53 \pm 14 \rightarrow \mathbf{149} \pm 6$	$69 \pm 7 \rightarrow \mathbf{145} \pm 5$
door-cloned-v1	$1 \rightarrow 20$	$0 \rightarrow \mathbf{102}$	$-0 \rightarrow -0$	$0 \pm 0 \rightarrow \mathbf{102} \pm 7$	$3 \pm 2 \rightarrow 50 \pm 15$	$0 \pm 0 \rightarrow \mathbf{102} \pm 5$	$0 \pm 0 \rightarrow 95 \pm 2$
hammer-cloned-v1	$1 \rightarrow 57$	$7 \rightarrow 125$	$0 \rightarrow 0$	$0 \pm 0 \rightarrow 128 \pm 29$	$4 \pm 2 \rightarrow 60 \pm 14$	$0 \pm 0 \rightarrow 127 \pm 17$	$0 \pm 0 \rightarrow \mathbf{134} \pm 4$
relocate-cloned-v1	$0 \rightarrow 0$	$1 \rightarrow 7$	$-0 \rightarrow -0$	$0 \pm 0 \rightarrow 2 \pm 2$	$-0 \pm 0 \rightarrow 5 \pm 3$	$0 \pm 1 \rightarrow \mathbf{62} \pm 8$	$0 \pm 0 \rightarrow 49 \pm 14$

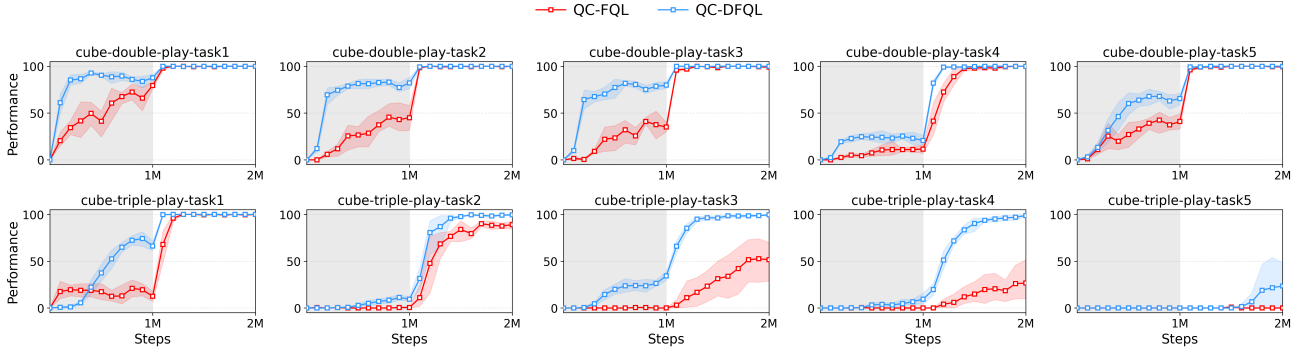


Figure 7. **Full QC-DFQL and QC-FQL results.** Experimental results show that QC-DFQL outperforms QC-FQL on nearly all tasks, indicating that DFQL, like FQL, can serve as a flow-matching-based module in offline/offline-to-online RL and can be integrated with other components to further improve performance.

Table 5. Full offline RL results. We report comprehensive offline RL results across all 73 tasks from the OGBench and D4RL suites. The default task for each environment is marked with an asterisk (*).

Task category	Gaussian policies			Diffusion policies			Flow policies				
	BC	IQL	ReBRAC	IDQL	SRPO	CAC	FAWAC	FBRAC	IFQL	FQL	DFQL
antmaze-large-navigate-singletask-task1-v0 (*)	0±0	48±9	91 ±10	0±0	0±0	42±7	1±1	70±20	24±17	80±8	93 ±4
antmaze-large-navigate-singletask-task2-v0	6±3	42±6	88 ±4	14±8	4±4	1±1	0±1	35±12	8±3	57±10	70±9
antmaze-large-navigate-singletask-task3-v0	29±5	72±7	51±18	26±8	3±2	49±10	12±4	83±15	52±17	93 ±3	97 ±3
antmaze-large-navigate-singletask-task4-v0	8±3	51±9	84±7	62±25	45±19	17±6	10±3	37±18	18±8	80±4	89 ±4
antmaze-large-navigate-singletask-task5-v0	10±3	54±22	90 ±2	2±2	1±1	55±6	9±5	76±8	38±18	83±4	90 ±4
antmaze-giant-navigate-singletask-task1-v0 (*)	0±0	0±0	27 ±22	0±0	0±0	0±0	0±0	0±1	0±0	4±5	11±7
antmaze-giant-navigate-singletask-task2-v0	0±0	1±1	16±17	0±0	0±0	0±0	0±0	4±7	0±0	9±7	48 ±13
antmaze-giant-navigate-singletask-task3-v0	0±0	0±0	34 ±22	0±0	0±0	0±0	0±0	0±0	0±0	0±1	4±4
antmaze-giant-navigate-singletask-task4-v0	0±0	0±0	5±12	0±0	0±0	0±0	0±0	9±4	0±0	14±23	56 ±11
antmaze-giant-navigate-singletask-task5-v0	1±1	19±7	49±22	0±1	0±0	0±0	0±0	6±10	13±9	16±28	77 ±7
humanoidmaze-medium-navigate-singletask-task1-v0 (*)	1±0	32±7	16±9	1±1	0±0	38±19	6±2	25±8	69 ±19	19±12	35±25
humanoidmaze-medium-navigate-singletask-task2-v0	1±0	41±9	18±16	1±1	1±1	47±35	40±2	76±10	85±11	94 ±3	93 ±4
humanoidmaze-medium-navigate-singletask-task3-v0	6±2	25±5	36±13	0±1	2±1	83 ±18	19±2	27±11	49±49	74±18	76±31
humanoidmaze-medium-navigate-singletask-task4-v0	0±0	0±1	15±16	1±1	1±1	5±4	1±1	1±2	1±1	3±4	32 ±19
humanoidmaze-medium-navigate-singletask-task5-v0	2±1	66±4	24±20	1±1	3±3	91±5	31±7	63±9	98 ±2	97 ±2	93 ±12
humanoidmaze-large-navigate-singletask-task1-v0 (*)	0±0	3±1	2±1	0±0	0±0	1±1	0±0	0±1	6±2	7±6	10 ±9
humanoidmaze-large-navigate-singletask-task2-v0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0
humanoidmaze-large-navigate-singletask-task3-v0	1±1	7±3	8±4	3±1	1±1	2±3	1±1	10±2	48 ±10	11±7	20±10
humanoidmaze-large-navigate-singletask-task4-v0	1±0	1±0	1±1	0±0	0±0	0±1	0±0	0±0	1±1	2 ±3	1±2
humanoidmaze-large-navigate-singletask-task5-v0	0±1	1±1	2±2	0±0	0±0	0±0	0±0	1±1	0±0	1±3	7 ±6
antsoccer-arena-navigate-singletask-task1-v0	2±1	14±5	0±0	44±12	2±1	1±3	22±2	17±3	61±25	77 ±4	77 ±7
antsoccer-arena-navigate-singletask-task2-v0	2±2	17±7	0±1	15±12	3±1	0±0	8±1	8±2	75±3	88 ±3	88 ±6
antsoccer-arena-navigate-singletask-task3-v0	0±0	6±4	0±0	0±0	0±0	8±19	11±5	16±3	14±22	61 ±6	51±10
antsoccer-arena-navigate-singletask-task4-v0 (*)	1±0	3±2	0±0	0±1	0±0	0±0	12±3	24±4	16±9	39 ±6	40 ±7
antsoccer-arena-navigate-singletask-task5-v0	0±0	2±2	0±0	0±0	0±0	0±0	9±2	15±4	0±1	36±9	38 ±13
cube-single-play-singletask-task1-v0	10±5	88±3	89±5	95 ±2	89±7	77±28	81±9	73±33	79±4	97 ±2	97 ±2
cube-single-play-singletask-task2-v0 (*)	3±1	85±8	92±4	96±2	82±16	80±30	81±9	83±13	73±3	97 ±2	99 ±2
cube-single-play-singletask-task3-v0	9±3	91±5	93±3	99 ±1	96 ±2	98 ±1	87±4	82±12	88±4	98 ±2	100 ±1
cube-single-play-singletask-task4-v0	2±1	73±6	92 ±3	93 ±4	70±18	91 ±2	79±6	79±20	79±6	94 ±3	94 ±4
cube-single-play-singletask-task5-v0	3±3	78±9	87±8	90 ±6	61±12	80±20	78±10	76±33	77±7	93 ±3	91 ±5
cube-double-play-singletask-task1-v0	8±3	27±5	45±6	39±19	7±6	21±8	21±7	47±11	35±9	61 ±9	62 ±20
cube-double-play-singletask-task2-v0 (*)	0±0	1±1	7±3	16±10	0±0	2±2	2±1	22±12	9±5	36±6	46 ±17
cube-double-play-singletask-task3-v0	0±0	0±0	4±1	17±8	0±1	3±1	1±1	4±2	8±5	22±5	46 ±16
cube-double-play-singletask-task4-v0	0±0	0±0	1±1	0±1	0±0	0±1	0±0	0±1	1±1	5±2	9 ±6
cube-double-play-singletask-task5-v0	0±0	4±3	4±2	1±1	0±0	3±2	2±1	2±2	17±6	19±10	27 ±20
scene-play-singletask-task1-v0	19±6	94±3	95 ±2	100 ±0	94±4	100 ±1	87±8	96 ±8	98 ±3	100 ±0	100 ±0
scene-play-singletask-task2-v0 (*)	1±1	12±3	50±13	33±14	2±2	50±40	18±8	46±10	0±0	76±9	97 ±3
scene-play-singletask-task3-v0	1±1	32±7	55±16	94 ±4	4±4	49±16	38±9	78±14	54±19	98 ±1	98 ±2
scene-play-singletask-task4-v0	2±2	0±1	3±3	4±3	0±0	0±0	6±1	4±4	0±0	5±1	9 ±13
scene-play-singletask-task5-v0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0
puzzle-3x3-play-singletask-task1-v0	5±2	33±6	97 ±4	52±12	89±5	97 ±2	25±9	63±19	94 ±3	90±4	94 ±8
puzzle-3x3-play-singletask-task2-v0	1±1	4±3	1±1	0±1	0±1	0±0	4±2	2±2	1±2	16±5	44 ±26
puzzle-3x3-play-singletask-task3-v0	1±1	3±2	3±1	0±0	0±0	0±0	1±0	1±1	0±0	10±3	33 ±13
puzzle-3x3-play-singletask-task4-v0 (*)	1±1	2±1	2±1	0±0	0±0	0±0	1±1	2±2	0±0	16±5	81 ±9
puzzle-3x3-play-singletask-task5-v0	1±0	3±2	5±3	0±0	0±0	0±0	1±1	2±2	0±0	16±3	75 ±20
puzzle-4x4-play-singletask-task1-v0	1±1	12±2	26±4	48±5	24±9	44±10	1±2	32±9	49±9	34±8	53 ±8
puzzle-4x4-play-singletask-task2-v0	0±0	7±4	12±4	14±5	0±1	0±0	0±1	5±3	4±4	16±5	17 ±6
puzzle-4x4-play-singletask-task3-v0	0±0	9±3	15±3	34±5	21±10	29±12	1±1	20±10	50 ±14	18±5	33±8
puzzle-4x4-play-singletask-task4-v0 (*)	0±0	5±2	10±3	26 ±6	7±4	1±1	0±0	5±1	21±11	11±3	14±6
puzzle-4x4-play-singletask-task5-v0	0±0	4±1	7±3	24 ±11	1±1	0±0	0±1	4±3	2±2	7±3	9±4
antmaze-umaze-v2	55	77	98	94	97	66±5	90±6	94 ±3	92±6	96 ±2	96 ±3
antmaze-umaze-diverse-v2	47	54	84	80	82	66±11	55±7	82±9	62±12	89 ±5	86 ±7
antmaze-medium-play-v2	0	66	90	84	81	49±24	52±12	77±7	56±15	78±7	81±7
antmaze-medium-diverse-v2	1	74	84	85	75	0±1	44±15	77±6	60±25	71±13	77±6
antmaze-large-play-v2	0	42	52	64	54	0±0	10±6	32±21	55±9	84 ±7	76±7
antmaze-large-diverse-v2	0	30	64	68	54	0±0	16±10	20±17	64±8	83 ±4	77±6
pen-human-v1	71	78	103	76±10	69±7	64±8	67±5	77±7	71±12	53±6	71±13
pen-cloned-v1	52	83	103	64±7	61±7	56±10	62±10	67±9	80±11	74±11	68±4
pen-expert-v1	110	128	152	140±6	134±4	103±9	118±6	119±7	139±5	142±6	140±9
door-human-v1	2	3	-0	6±2	3±3	5±2	2±1	4±2	7 ±2	0±0	0±1
door-cloned-v1	-0	3	0	0±0	0±0	1±0	0±1	0±0	2±2	2±1	0±0
door-expert-v1	105	107	106	105 ±1	105 ±0	98±3	103 ±1	104 ±1	104 ±2	104 ±1	104 ±1
hammer-human-v1	3	2	0	2±1	1±1	2±0	2±1	2±1	3 ±1	1±1	0±0
hammer-cloned-v1	1	2	5	2±1	2±1	1±1	1±0	2±1	2±1	11 ±9	6±3
hammer-expert-v1	127	129	134	125±4	127±0	92±11	118±3	119±9	117±9	125±3	123±3
relocate-human-v1	0	0	0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0	0 ±0
relocate-cloned-v1	-0	0	2	-0±0	-0±0	-0±0	-0±0	1±1	-0±0	-0±0	0±0
relocate-expert-v1	108	106	108	107 ±1	106 ±2	93±6	105 ±3	105 ±2	104 ±3	107 ±1	108 ±2
visual-cube-single-play-singletask-task1-v0	—	70±12	83±6	—	—	—	—	55±8	49±7	81±12	96 ±3
visual-cube-double-play-singletask-task1-v0	—	34 ±23	4±4	—	—	—	—	6±2	8±6	21±11	27±21
visual-scene-play-singletask-task1-v0	—	97 ±2	98 ±4	—	—	—	—	46±4	86±10	98 ±3	100 ±1
visual-puzzle-3x3-play-singletask-task1-v0	—	7±15	88±4	—	—	—	—	7±2	100 ±0	94±1	99 ±2
visual-puzzle-4x4-play-singletask-task1-v0	—	0±0	26±6	—	—	—	—	0±0	8±15	33±6	40 ±9

F. Experimental Details

F.1. Environments

Our implementation of DFQL and baselines is based on the JAX framework (Bradbury et al., 2018) and builds on the official OGBench reference implementations.

OGBench (Park et al., 2025a):

Our main evaluation suite is OGBench. We select 10 environments, covering 50 state-based tasks and 5 pixel-based tasks. Since OGBench is primarily designed for goal-conditioned offline RL, we follow its *single-task* setting (environments with the “-singletask” suffix) to benchmark standard reward-maximizing offline RL methods.

In OGBench, each environment includes five evaluation goals, corresponding to five tasks (“-singletask-task1” to “-singletask-task5”), with “-singletask” (no suffix) denoting the default task. We adopt the single-task reward/termination specification from the benchmark (Park et al., 2025a). Concretely, rewards are semi-sparse and depend on goal progress: for locomotion-style tasks that typically contain a single subtask (e.g., “reach the goal”), the reward is either -1 or 0 ; for manipulation-style tasks with multiple subtasks (e.g., “open the drawer”, “turn the first button’s color blue”), rewards lie in $[-n_{\text{task}}, 0]$, where n_{task} is the number of subtasks (up to 16 in our selected environments). Episodes terminate upon completing the final goal, consistent with the official protocol (Park et al., 2025a).

We evaluate the following datasets (five tasks per environment), resulting in ten state-based datasets and five pixel-based datasets:

- State-based datasets:
 - antmaze-large-navigate-v0
 - antmaze-giant-navigate-v0
 - humanoidmaze-medium-navigate-v0
 - humanoidmaze-large-navigate-v0
 - antsoccer-arena-navigate-v0
 - cube-single-play-v0
 - cube-double-play-v0
 - scene-play-v0
 - puzzle-3x3-play-v0
 - puzzle-4x4-play-v0
- Pixel-based datasets:
 - visual-cube-single-play-v0
 - visual-cube-double-play-v0
 - visual-scene-play-v0
 - visual-puzzle-3x3-play-v0
 - visual-puzzle-4x4-play-v0

For evaluation, we report binary task success rates (percentages). Due to computational constraints, for each pixel-based environment we evaluate only the first task (“-singletask-task1”).

D4RL (Fu et al., 2020):

To facilitate comparison with prior results, we additionally evaluate on 18 standard tasks from D4RL, including 6 antmaze tasks and 12 adroit tasks:

- D4RL Antmaze tasks:
 - antmaze-umaze-v2
 - antmaze-umaze-diverse-v2
 - antmaze-medium-play-v2

- antmaze-medium-diverse-v2
- antmaze-large-play-v2
- antmaze-large-diverse-v2
- D4RL Adroit tasks:
 - pen-human-v1, pen-cloned-v1, pen-expert-v1
 - door-human-v1, door-cloned-v1, door-expert-v1
 - hammer-human-v1, hammer-cloned-v1, hammer-expert-v1
 - relocate-human-v1, relocate-cloned-v1, relocate-expert-v1

Following the standard D4RL evaluation protocol (Fu et al., 2020), we report binary success rates for antmaze tasks and normalized returns for adroit tasks. Note that while the D4RL antmaze tasks share a similar high-level objective with OGBench navigation tasks, they differ in layout, datasets, and evaluation details.

F.2. Hyperparameter Tuning

Complete hyperparameter lists are provided in the following tables (Tables 6, 7, and 8).

Table 6. Hyperparameters for DFQL.

Hyperparameter	Value
Learning rate	0.0003
Optimizer	Adam
Gradient steps	1000000 (default), 500000 (D4RL, pixel-based OGBench)
Minibatch size	256
MLP dimensions	[512, 512, 512, 512]
Nonlinearity	GELU & Tanh (additional on the last layer)
Target network smoothing coefficient	0.005
Discount factor γ	0.99 (default), 0.995 (antmaze-giant, humanoidmaze, antsoccer)
Image augmentation probability	0.5
Flow steps	10
Clipped double Q-learning	False (default), True (adroit, antmaze- $\{\text{large, giant}\}$ -navigate)
coefficient α	Tables 7 and 8

Table 7. Task-specific hyperparameters for offline RL. We perform cross-task adjustments for the hyperparameter α , which is individually tuned for each task. But in OGBench, we tune it only on the default task (denoted by (*)) and apply the resulting best hyperparameters to the other four tasks.

Task	α
antmaze-large-navigate-singletask-task1-v0 (*)	20
antmaze-large-navigate-singletask-task2-v0	20
antmaze-large-navigate-singletask-task3-v0	20
antmaze-large-navigate-singletask-task4-v0	20
antmaze-large-navigate-singletask-task5-v0	20
antmaze-giant-navigate-singletask-task1-v0 (*)	10
antmaze-giant-navigate-singletask-task2-v0	10
antmaze-giant-navigate-singletask-task3-v0	10
antmaze-giant-navigate-singletask-task4-v0	10
antmaze-giant-navigate-singletask-task5-v0	10
humanoidmaze-medium-navigate-singletask-task1-v0 (*)	10
humanoidmaze-medium-navigate-singletask-task2-v0	10
humanoidmaze-medium-navigate-singletask-task3-v0	10
humanoidmaze-medium-navigate-singletask-task4-v0	10
humanoidmaze-medium-navigate-singletask-task5-v0	10
humanoidmaze-large-navigate-singletask-task1-v0 (*)	10
humanoidmaze-large-navigate-singletask-task2-v0	10
humanoidmaze-large-navigate-singletask-task3-v0	10
humanoidmaze-large-navigate-singletask-task4-v0	10
humanoidmaze-large-navigate-singletask-task5-v0	10
antsoccer-arena-navigate-singletask-task1-v0	10
antsoccer-arena-navigate-singletask-task2-v0	10
antsoccer-arena-navigate-singletask-task3-v0	10
antsoccer-arena-navigate-singletask-task4-v0 (*)	10
antsoccer-arena-navigate-singletask-task5-v0	10
cube-single-play-singletask-task1-v0	1
cube-single-play-singletask-task2-v0 (*)	1
cube-single-play-singletask-task3-v0	1
cube-single-play-singletask-task4-v0	1
cube-single-play-singletask-task5-v0	1
cube-double-play-singletask-task1-v0	2
cube-double-play-singletask-task2-v0 (*)	2
cube-double-play-singletask-task3-v0	2
cube-double-play-singletask-task4-v0	2
cube-double-play-singletask-task5-v0	2
scene-play-singletask-task1-v0	1
scene-play-singletask-task2-v0 (*)	1
scene-play-singletask-task3-v0	1
scene-play-singletask-task4-v0	1
scene-play-singletask-task5-v0	1
puzzle-3x3-play-singletask-task1-v0	3.3
puzzle-3x3-play-singletask-task2-v0	3.3
puzzle-3x3-play-singletask-task3-v0	3.3
puzzle-3x3-play-singletask-task4-v0 (*)	3.3
puzzle-3x3-play-singletask-task5-v0	3.3
puzzle-4x4-play-singletask-task1-v0	0.2
puzzle-4x4-play-singletask-task2-v0	0.2
puzzle-4x4-play-singletask-task3-v0	0.2
puzzle-4x4-play-singletask-task4-v0 (*)	0.2
puzzle-4x4-play-singletask-task5-v0	0.2
antmaze-umaze-v2	20
antmaze-umaze-diverse-v2	20
antmaze-medium-play-v2	20
antmaze-medium-diverse-v2	20
antmaze-large-play-v2	33
antmaze-large-diverse-v2	33
pen-human-v1	0.033
pen-cloned-v1	0.033
pen-expert-v1	0.2
door-human-v1	0.01
door-cloned-v1	0.01
door-expert-v1	0.01
hammer-human-v1	0.033
hammer-cloned-v1	0.033
hammer-expert-v1	0.0033
relocate-human-v1	0.033
relocate-cloned-v1	0.01
relocate-expert-v1	0.01
visual-cube-single-play-singletask-task1-v0	0.33
visual-cube-double-play-singletask-task1-v0	0.33
visual-scene-play-singletask-task1-v0	1
visual-puzzle-3x3-play-singletask-task1-v0	1
visual-puzzle-4x4-play-singletask-task1-v0	0.33

Table 8. **Task-specific hyperparameters for offline-to-online RL.** We individually tune these hyperparameters for each task.

Task	α
humanoidmaze-medium-navigate-singletask-v0	10
antsoccer-arena-navigate-singletask-v0	10
cube-double-play-singletask-v0	2
scene-play-singletask-v0	1
puzzle-4x4-play-singletask-v0	0.2
antmaze-umaze-v2	20
antmaze-umaze-diverse-v2	20
antmaze-medium-play-v2	20
antmaze-medium-diverse-v2	20
antmaze-large-play-v2	33
antmaze-large-diverse-v2	33
pen-cloned-v1	0.1
door-cloned-v1	0.1
hammer-cloned-v1	0.1
relocate-cloned-v1	0.2